

Digital computers

Digital computers are machines that process, store, and manipulate information using discrete values, typically represented as binary digits (bits), which are combinations of 0s and 1s. Unlike analog computers, which use continuous signals, digital computers operate on data in a digital format, making them highly precise, versatile, and capable of performing complex computations. They form the backbone of modern technology, powering everything from smartphones and laptops to servers and supercomputers.

Definition and Core Concept

A digital computer is an electronic device that accepts input data, processes it according to a set of instructions (called a program), and produces output in a usable form. The "digital" aspect refers to its use of discrete signals—binary code (0s and 1s)—to represent data. These binary digits are manipulated using logical operations based on Boolean algebra.

For example:

- The letter "A" might be represented as 01000001 in binary (ASCII encoding).
- Numbers, images, sounds, and even instructions are all converted into binary for processing.

Key Components

Digital computers consist of hardware and software working together. Here's a detailed look at their components:

Hardware

- **Central Processing Unit (CPU):** Often called the "brain" of the computer, the CPU executes instructions from programs by performing the basic operations of fetch, decode, and execute. Modern CPUs have multiple cores to handle tasks in parallel.
- **Memory:**
 - **RAM (Random Access Memory):** Temporary storage for data and instructions the CPU needs while running programs. It's volatile, meaning it clears when the computer powers off.
 - **ROM (Read-Only Memory):** Non-volatile memory that stores firmware or boot-up instructions.

- **Storage:** Long-term data storage devices like hard drives (HDDs), solid-state drives (SSDs), or flash memory, where programs and files are kept.
- **Input Devices:** Tools like keyboards, mice, or scanners that send data to the computer.
- **Output Devices:** Displays (monitors), printers, or speakers that present processed results.
- **Motherboard:** The main circuit board connecting all components, including buses for data transfer.

Software

- **System Software:** Operating systems (e.g., Windows, Linux, macOS) manage hardware and provide a platform for other software.
- **Application Software:** Programs like web browsers, word processors, or games that perform specific tasks for users.

How Digital Computers Work

Digital computers operate through a cycle known as the Instruction Cycle:

1. **Input:** Data and instructions enter via input devices (e.g., typing a command).
2. **Storage:** Data is stored in memory or on a storage device.
3. **Processing:** The CPU processes the data by executing programmed instructions, often involving arithmetic (e.g., addition) or logical operations (e.g., comparisons).
4. **Output:** Results are sent to output devices (e.g., displaying text on a screen).

Characteristics of Digital Computers

- **Accuracy:** They perform calculations with high precision, limited only by hardware constraints or programming errors.
- **Speed:** Modern digital computers can execute billions of operations per second (measured in GHz or flops for supercomputers).
- **Versatility:** They can handle diverse tasks—calculations, gaming, data analysis—by running different programs.
- **Storage Capacity:** They can store vast amounts of data, from kilobytes to petabytes, depending on the system.
- **Programmability:** Users can change a computer's behavior by loading new software.

Types of Digital Computers

Digital computers vary in size, power, and purpose:

- **Microcomputers:** Personal devices like PCs, laptops, and smartphones (e.g., your device reading this).
- **Minicomputers:** Mid-sized systems for small businesses or specific tasks (less common today).
- **Mainframes:** Large, powerful systems for handling massive data processing (e.g., banking transactions).
- **Supercomputers:** Extremely fast machines for complex simulations (e.g., weather forecasting, quantum research).

Advantages

- **Precision:** No signal degradation like in analog systems.
- **Scalability:** Easily upgraded with more memory, storage, or processing power.
- **Reproducibility:** Results are consistent when the same inputs and instructions are used.
- **Connectivity:** Digital computers can communicate over networks (e.g., the internet).

Limitations

- **Complexity:** Designing and programming them requires significant expertise.
- **Power Dependency:** They need a constant power supply to function.
- **Data Conversion:** Real-world analog data (e.g., sound waves) must be digitized, which can introduce minor errors (e.g., quantization noise).

Historical Context

- **Early Beginnings:** The concept emerged with Charles Babbage's Analytical Engine (1830s), though it was mechanical. The first electronic digital computer, ENIAC (1945), used vacuum tubes.
- **Evolution:** Transistors (1950s), integrated circuits (1960s), and microprocessors (1970s) made computers smaller, faster, and more affordable.
- **Today:** Quantum computing experiments hint at the next frontier, though classical digital computers dominate.

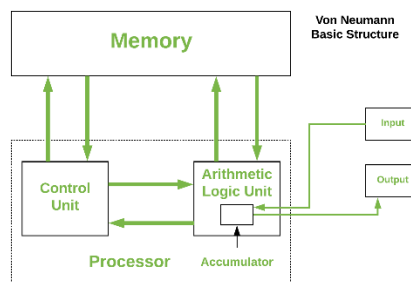
Von Neumann architecture:

Jhon Von-Neumann proposed his computer architecture design in 1945 which was later known as Von-Neumann Architecture.

Von Neumann architecture is based on the stored-program computer concept, where data and instruction are stored in the same memory simplifying hardware design and enabling general-purpose computing. This design is still used in most computers produced today.

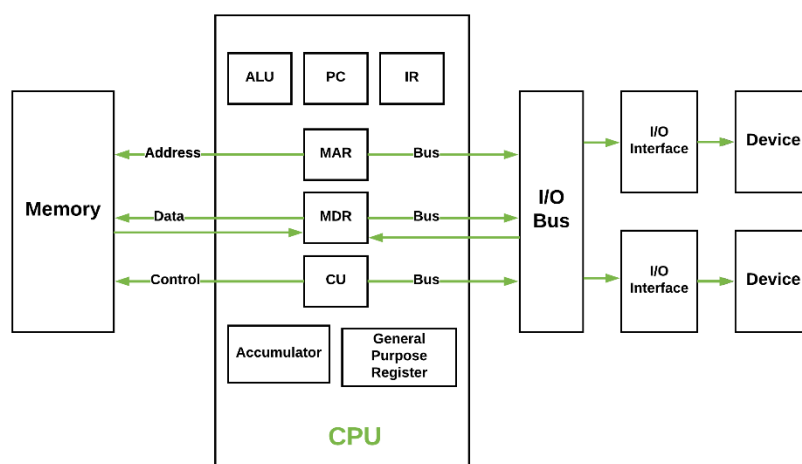
It consisted of a Control Unit, Arithmetic, and Logical Memory Unit (ALU), Registers and Inputs/Outputs.

Block Diagram:



Components of Von-Neumann Model:

- Central Processing Unit
- Buses
- Memory Unit



Central Processing Unit:

The part of the computer that performs the bulk of data processing operations is called the Central Processing Unit and is referred to as the CPU.

The Central Processing Unit can also be defined as an electric circuit responsible for executing the instructions of a computer program.

The CPU performs a variety of functions dictated by the type of instructions that are incorporated in the computer.

It has following major components:

1. Control Unit (CU)
2. Arithmetic and Logic Unit (ALU)
3. variety of Registers

Control Unit (CU)

The Control Unit of a computer system controls the operations of components like ALU, memory and input/output devices.

It directs the operation of the computer by fetching instructions from memory, decoding them, and coordinating execution.

Arithmetic	and	Logic	Unit	(ALU)
Performs arithmetic operations (e.g., addition, subtraction) and logical operations (e.g. AND, OR comparisons).				

Registers

Registers are small, high-speed memory inside the CPU that store data temporarily while processing.

- **Accumulator (ACC):** Holds the results of calculations done by the ALU. It acts as temporary storage for arithmetic and logical operations.
- **Program Counter (PC):** Keeps track of the address of the next instruction to be executed. It sends this address to the Memory Address Register (MAR).
- **Memory Address Register (MAR):** Stores the memory location of instructions or data that need to be fetched or saved.
- **Memory Data Register (MDR):** Holds the data or instructions fetched from memory before processing or saving.
- **Instruction Register (IR):** Stores the most recently fetched instruction while it is being decoded and executed.

- **Instruction Buffer Register (IBR):** Temporarily holds instructions that are not executed immediately.

Buses

Buses transfer data between different parts of a computer, connecting the CPU, memory, and other components. Types:

- **Data Bus:** Transfers actual data between the CPU, memory, and input/output devices.
- **Address Bus:** Sends the location (address) of the data, not the data itself, between the CPU and memory.
- **Control Bus:** Sends control signals from the CPU to manage and coordinate computer operations.

Input/Output Devices

Data is read into main memory from the input device or secondary storage under the control of CPU input instruction.

Output devices are used to output information from a computer. If some results are evaluated by the computer and it is stored in the computer, then with the help of output devices, we can present them to the user.

Memory Unit

A memory unit is a collection of storage cells that store and transfer data. It holds binary data in groups called words, which have a specific number of bits.

Types of Memory in a Computer:

- RAM (Random Access Memory)
 - Stores data that the computer is currently using.
 - Works only when the computer is on.
 - It is volatile, meaning data is lost when the computer is turned off.
- ROM (Read-Only Memory)
 - Stores permanent data and startup instructions (BIOS).
 - Unlike RAM, it is non-volatile, meaning data stays even when the computer is off.

Von Neumann Bottleneck

The Von Neumann bottleneck is a limitation in traditional computers where:

1. Instructions are processed one at a time.
2. They are executed in order (sequentially).

This slows down the CPU because it can't process multiple instructions at once. Even if we add more cache, RAM, or faster components, the CPU's design still limits performance.

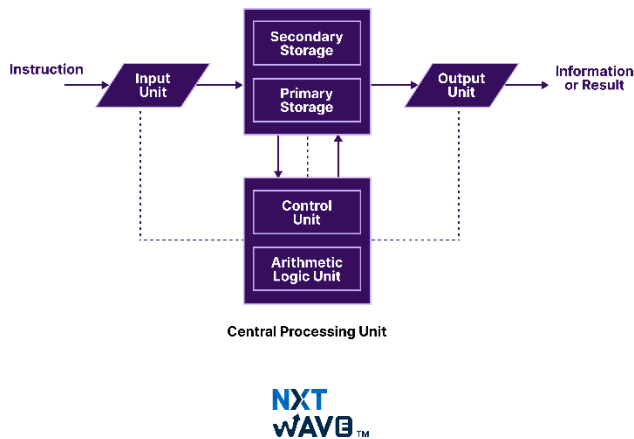
Basic Organization of Computer

The basic organization of a computer system refers to the structure and arrangement of its core components that work together to perform computational tasks. A computer system is made up of both hardware and software, where hardware refers to the physical devices, and software refers to the programs or instructions followed to execute specific tasks and operate computers.

At a fundamental level, a computer is a system that takes input, processes it, stores the data temporarily or permanently, and produces output. The organization of a computer is based on Von Neumann Architecture, proposed by John von Neumann in 1945. This architecture divides the computer system into several functional units that work together to execute tasks. These basic components are still used in modern computers, even as their complexity and efficiency have grown significantly.

The key elements in the basic organization of a computer system are:

- Input Unit
- Output Unit
- Control Unit (CU)
- Arithmetic Logic Unit (ALU)
- Memory Unit (Storage)



Functional Components of a Computer

Here are the functional components of a computer:

1. Input Unit

The input unit is responsible for receiving data and instructions from the external environment and converting them into a format that the computer can process. It acts as a bridge between the outside world and the computer system.

Common input devices include:

- Keyboard, Mouse, Scanner, Microphone, Joystick, Webcam

2. Output Unit

The output unit provides processed data from the computer system to the external world in a human-readable form. It is the counterpart to the input unit and displays the results of computations.

Common output devices include Monitor, Printer, Speakers

3. Control Unit (CU)

The Control Unit (CU) is often described as the 'brain' or 'central nervous system' of the computer. It directs the operations of the entire computer system, including managing the flow of data and instructions.

4. Arithmetic Logic Unit (ALU)

The Arithmetic Logic Unit (ALU) is responsible for performing all the mathematical and logical operations in a computer system. This includes operations like:

Arithmetic operations: Addition, subtraction, multiplication, division.

Logical operations: AND, OR, NOT, XOR.

5. Memory Unit (Storage)

The Memory Unit is used to store data and instructions that are necessary for execution by the CPU. The memory is broadly categorized into two types:

- **Primary Memory (Internal Memory):** This includes RAM and ROM, which are directly accessible by the CPU.
- **Secondary Memory:** This is used for permanent storage and includes devices like hard drives, optical disks, and magnetic tapes.

Difference between Hardware and Software

Here is the comparison of Hardware and Software:

Hardware	Software
It defines the physical components that process data.	It is a set of instructions that tell the computer to perform tasks.
It is manufactured using electronic and mechanical materials.	It is developed by writing code in programming languages.
It cannot function without software.	The software needs hardware to run and execute.
The hardware is built using metals, plastics, and other materials.	The process of software is developed using programming languages and algorithms.
It is a physical device or sensible gadget (e.g., keyboard, mouse).	It exists in the form of code and files.
The durability of the device lasts for a long time (e.g., hard drive failure, keyboard keys wear out).	It does not physically degrade but may have bugs or glitches over time.

Hardware is not affected by viruses.	Software is vulnerable to viruses, malware, and security flaws.
It cannot be transferred electronically through a network.	It can be easily transferred via a network, cloud, or USB drive.
Hardware operates at machine-level language (binary).	Software is written in high-level programming languages like Java, Python, and C++.
If damaged, hardware must be physically replaced.	Damaged software can be reinstalled or restored from a backup.
The damage caused by physical factors like dust, overheating, humidity, or physical damage.	The damage caused by software bugs, crashes, version conflicts, or overloading.
Examples: Keyboard, mouse, CPU, RAM, hard disk, motherboard, etc.	Examples: MS Word, Excel, Photoshop, MySQL, video games, operating systems, etc.

Computer Generations

Generations of Computer	Time-Period	Evolving Hardware
First Generation	1940s – 1950s	Vacuum Tube Based
Second Generation	1950s – 1960s	Transistor Based

Third Generation	1960s – 1970s	Integrated Circuit Based
Fourth Generation	1970s – Present	Microprocessor Based
Fifth Generation	Present – Future	Artificial Intelligence Based

First Generation

First-generation computers used vacuum tubes, which were fragile glass devices. They were large, heavy, and unreliable, with no operating system and low-level programming. These computers required entire rooms and consumed a lot of electricity. They were used for calculations, storage, and control. Punch cards were used for external storage, and machine and assembly languages were developed.

Here are some key examples of first-generation computers:

- **ENIAC (Electronic Numerical Integrator and Computer)** – Built by J. Presper Eckert and John V. Mauchly, it was a general-purpose computer with 18,000 vacuum tubes. It was large and cumbersome.
- **EDVAC (Electronic Discrete Variable Automatic Computer)** – Designed by John von Neumann, it could store both data and instructions, improving processing speed.
- **UNIVAC (Universal Automatic Computer)** – Developed in 1952 by Eckert and Mauchly, it was one of the first commercially available computers.

Second Generation Computers

Second-generation computers used the technology of transistors rather than bulky vacuum tubes. Another feature was the core storage. A transistor may be a device composed of semiconductor material that amplifies a sign or opens or closes a circuit.

Transistors were invented in Bell Labs. The use of transistors made it possible to perform powerfully and with due speed. It reduced the dimensions and price and thankfully the warmth too, which was generated by vacuum tubes. Central Processing Unit (CPU), memory, programming language, and input, and output units also came into the force within the second generation.

Programming shifted from low-level to high-level languages, making coding easier for programmers. Popular languages from this era included FORTRAN (1956), ALGOL (1958), and COBOL (1959).

Third Generation Computers

The third generation introduced integrated circuits (ICs), replacing large transistors. Multiple transistors were placed on silicon chips, also known as semiconductors. These computers were faster, more reliable, and more efficient due to IC technology.

The computer programs were designed to make the machine work. Operating system was a program designed to handle a machine completely. Because of the operating system machine could execute multiple jobs simultaneously. Integrated circuits were used to replace many transistors used in the second generation.

A single IC has many transistors, registers, and capacitors built on one thin slice of silicon. The value size was reduced and memory space and dealing efficiency were increased during this generation. Programming was now wiped-out Higher-level languages like BASIC (Beginners All-purpose Symbolic Instruction Code). Minicomputers find their shape during this era.

Fourth Generation Computers

In 1971 First microprocessors were used, the large-scale of integration LSI circuits built on one chip called microprocessors. The advantage of this technology is that one microprocessor can contain all the circuits required to perform arithmetic, logic, and control functions on one chip. LSI placed thousands of transistors onto a single chip.

Computers using microchips were called microcomputers, offering smaller sizes with greater capacity. **Very Large-Scale Integration (VLSI)** circuits replaced LSI circuits, fitting hundreds of thousands of transistors onto a single silicon chip, known as a **microprocessor**.

The Intel 4004 chip (1971) integrated the CPU, memory, and I/O controls into one chip, drastically reducing computer size. Technologies like multiprocessing, multiprogramming, time-sharing, virtual memory, and faster operating speeds made computers more user-friendly. This generation also introduced personal computers (PCs) and computer networks.

Fifth Generation Computers

The technology behind the fifth generation of computers is AI. It allows computers to behave like humans. It is often seen in programs like voice recognition, area of medicine, and entertainment. Within the field of game playing also it's shown remarkable performance where computers can beat human competitors.

The speed is the highest, size is the smallest and area of use has remarkably increased within the fifth-generation computers. Though not a hundred percent AI has been achieved to date but keeping in sight the present developments, it is often said that this dream also will become a reality very soon.

To summarize the features of varied generations of computers, it is often said that a big improvement has been seen so far because of the speed and accuracy of functioning care, but if we mention the dimensions, it's been small over the years. The value is additionally diminishing, and reliability is increasing.

Instruction Codes

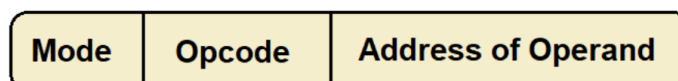
Instruction codes in computer architecture are concise groups of bits that instruct computers to execute a specific task operation. Instruction codes and addresses are unique to each computer. The operation code (opcode) in computer instructions is like a secret code that commands the computer what to do such as addition, subtraction, shift, complement, and control flow instructions.

Generally, we can categorize instruction codes and addresses into operation codes (Opcodes) and addresses.

What are Instruction Codes and Operands in Computer Architecture?

Instruction codes are bits that instruct the computer to execute a specific operation. An instruction comprises groups called fields. These fields include:

- The operation code (Opcode)
- The Address field
- The Mode field



Structure of an Instruction Code

The instruction code is also known as an instruction set. The structure of an instruction code can vary. It depends on the architecture of the processor but generally consists of the following parts:

- **Opcode:** The opcode (Operation code) represents the operation that the processor must perform. It might indicate that the instruction is an arithmetic operation such as addition,

subtraction, multiplication, or division.

- **Operand(s):** The operand(s) represents the data that the operation must be performed on. This data can take various forms, depending on the processor's architecture. It might be a register containing a value, a memory address pointing to a location in memory where the data is stored, or a constant value embedded within the instruction itself.
- **Addressing mode:** The addressing mode determines how an operand is accessed. It can be:
 - Direct addressing – The operand is a memory address.
 - Indirect addressing – The operand is a memory address stored in a register.
 - Immediate addressing – The operand is a constant value within the instruction.
- **Prefixes or modifiers:** Some instruction sets may include additional prefixes or modifiers that modify the behavior of the instruction. For example, they may specify that the operation should be performed only if a specific condition is met or that the instruction should be executed repeatedly until a specific condition is met.

Types of Instruction Code

There are various types of instruction codes. They are classified based on the number of operands, the type of operation performed, and the addressing modes used. The following are some common types of instruction codes:

1. **One-operand instructions:** These instructions have one operand and operate on that operand. For example, the "neg" instruction in the x86 assembly language negates the value of a single operand.
2. **Two-operand instructions:** These instructions have two operands and perform an operation involving both. For example, the "add" instruction in x86 assembly language adds two operands together.
3. **Three-operand instructions:** These instructions have three operands and perform an operation that involves all three operands. For example, the "fma" (fused multiply-add) instruction in some processors multiplies two operands together, adds a third operand,

and stores the result in a fourth operand.

4. **Data transfer instructions:** These instructions move data between memory and registers or between registers. For example, the "mov" instruction in the x86 assembly language moves data from one location to another.
5. **Control transfer instructions:** These instructions change the flow of program execution by modifying the program counter. For example, the "jmp" instruction in the x86 assembly language jumps to a different location in the program.
6. **Arithmetic instructions:** These instructions perform mathematical operations on operands.
7. **Logical instructions:** These instructions perform logical operations on operands. For example, the "and" instruction in x86 assembly language performs a bitwise AND operation on two operands.
8. **Comparison instructions:** These instructions compare two operands and set a flag based on the result. For example, the "cmp" instruction in x86 assembly language compares two operands and sets a flag indicating whether they are equal, greater than, or less than.
9. **Floating-point instructions:** These instructions perform arithmetic and other operations on floating-point numbers. For example, the "fadd" instruction in the x86 assembly language adds two floating-point numbers together.

Opcodes: An opcode is a collection of bits representing the basic operations, including add, subtract, multiply, complement, and shift. The number of bits required for an opcode depends on the number of operations a computer supports. For 2^n operations, at least n bits are needed for the opcode. These operations are performed on data stored in processor registers or memory.

Types of Opcodes

There are three different types of instruction codes on the main computer. The instruction's operation code (opcode) is 3 bits long, and the remaining 13 bits are determined by the operation code encountered.

There are three types of formats:

1. **Memory Reference Instruction:** It specifies the address with 12 bits and the addressing mode with 1 bit (I). For direct addresses, I equal 0, while for indirect addresses, I equal 1.
2. **Register Reference Instruction:** The opcode 111 with a 0 in the leftmost bit of the instruction recognizes these instructions. The remaining 12 bits specify the procedure to be carried out.
3. **Input-Output Instruction:** The operation code 111 with a 1 in the leftmost bit of instruction recognizes these instructions. The input-output action is specified using the remaining 12 bits.

Address

An instruction code contains address bits that specify where an instruction is stored. Instead of directly storing an address, an instruction can have:

- **Immediate operand** – The instruction contains the actual data.
- **Direct address** – The second part of the instruction holds the memory address of the operand.
- **Indirect address** – The second part points to another address that holds the operand.

A **single bit** can indicate whether the instruction uses **direct or indirect addressing**.

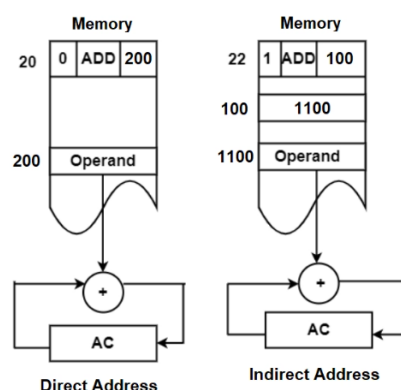
Addressing Modes

We can mainly describe the address field for instruction in the following ways:

- **Direct Addressing** – Uses the address of the operand.
- **Indirect Addressing** – Enables the address as a pointer to the operand.
- **Immediate operand** – The second part of the instruction code specifies an operand.

Effective Address (EA) is the address of the operand or the target address. It defines the address that we can execute as a target address for a branch-type instruction or the address

we can use directly to create an operand for a computation-type instruction without any changes.



Stored Program Organization

The simplest way to organize a computer system is by using a single processor register and an instruction code format with two parts. The first part specifies the operation to be performed, known as the **opcode**, while the second part specifies the memory address where the required data, called the **operand**, is stored.

In this system, the control unit determines where to find the operand in memory based on the given address. Once the operand is retrieved from memory, it is combined with the data already stored in the processor register to perform the required operation.

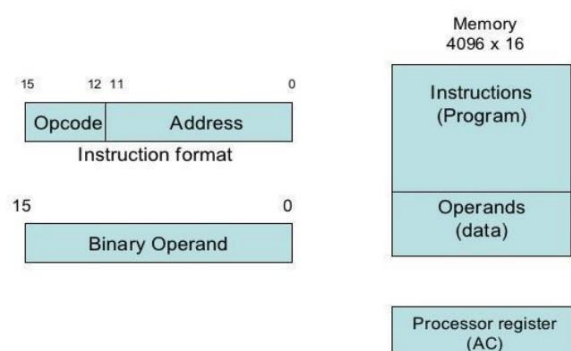


Figure 2.1: Stored Program Organization

In a stored-program architecture, instructions are stored in one section of memory, while data is stored separately in another section.

If the computer has a memory unit with 4096 words, it needs 12 bits to represent an address because $2^{12} = 4096$.

Each instruction is stored as a 16-bit word in memory, where:

- 4 bits are used for the operation code (opcode) (allowing 16 possible operations).
- 12 bits are used for the operand's memory address.

When an instruction is executed, the control unit first fetches a 16-bit instruction from the program memory. It then reads the 12-bit address in the instruction to locate the required operand in the data memory. After retrieving the operand, the control unit performs the operation specified by the opcode.

Early computers often used a single processor register called the Accumulator (AC) for calculations. Some operations, such as clearing AC, complementing AC, and incrementing AC, do not require an operand from memory because they work directly on the data stored in AC. In such cases, the 12-bit address section of the instruction is not needed and can be used for other functions instead.

Computer Registers

Registers are a type of computer memory used to quickly accept, store, and transfer data and instructions that are being used immediately by the CPU. The registers used by the CPU are often termed as Processor registers.

A processor register may hold an instruction, a storage address, or any data (such as bit sequence or individual characters).

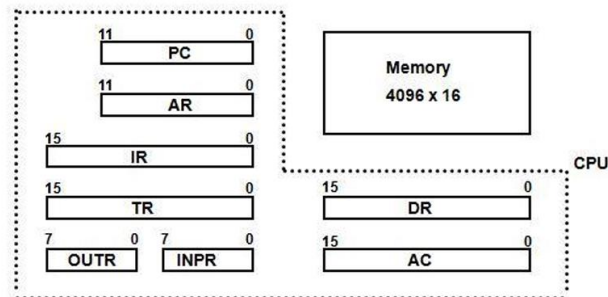
The computer needs processor registers for manipulating data and a register for holding a memory address. The register holding the memory location is used to calculate the address of the next instruction after the execution of the current instruction is completed.

Following is the list of some of the most common registers used in a basic computer:

Register Symbol	Bits	Register Name	Function
DR	16	Data register	Holds memory operand
AR	12	Address register	Holds address for memory
AC	16	Accumulator	Processor register
IR	16	Instruction register	Holds instruction code
PC	12	Program counter	Holds address of instruction
TR	16	Temporary register	Holds temporary data
INPR	8	Input register	Holds input character
OUTR	8	Output register	Holds output character

Table 2.1: List of Registers for Basic Computer

The following image shows the register and memory configuration for a basic computer.



The Memory unit has a capacity of 4096 words, and each word contains 16 bits.

- **The Data Register (DR)** contains 16 bits which hold the operand read from the memory location.
- **The Memory Address Register (MAR)** contains 12 bits which hold the address for the memory location.
- **The Program Counter (PC)** also contains 12 bits which hold the address of the next instruction to be read from memory after the current instruction is executed.
- **The Accumulator (AC)** register is a general-purpose processing register.
- The instruction read from memory is placed in the **Instruction register (IR)**.
- **The Temporary Register (TR)** is used for holding the temporary data during the processing.
- **The Input Registers (IR)** holds the input characters given by the user.
- **The Output Registers (OR)** holds the output after processing the input data.

Computer Instructions

Computer instructions are a set of machine language instructions that a particular processor understands and executes. A computer performs tasks on the basis of the instruction provided. Instructions are the order which is given by the user to the computer to perform some specific function desired. Mostly the codes of the instruction are written in binary language, i.e: 0's and 1's. ADD, JUMP, LOAD, etc. are some of the instructions used in processors.

An instruction comprises of groups called fields. These fields include:

- The operation code (Opcode) field which specifies the operation to be performed.
- The Address field which contains the location of the operand, i.e., register or memory location.
- The Mode field which specifies how the operand will be located.



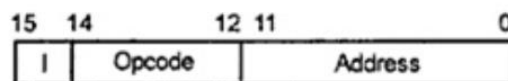
A basic computer has three instruction code formats which are:

website: www.notedup.in

1. Memory - reference instruction
2. Register - reference instruction
3. Input-Output instruction

1. Memory Reference Instruction

These instructions are mainly used to perform operations on operands that are located in memory.



(a) Memory reference instruction

From bit 0 to bit 11, represent the address field that specifies the operand's address. From bit 12 to bit 14 represent the opcode field which consists of 7 operations as 3 bits are used to specify it. The 15th bit specifies the indirect bit. If $I=0$, this means direct address and $I = 1$ means indirect address.

Instruction	I=0	I=1	Description
AND	0XXX	8XXX	Performs AND operation between memory and accumulator.
ADD	1XXX	9XXX	Performs add operation between memory and accumulator.
LDA	2XXX	AXXX	Load accumulator in memory
STA	3XXX	BXXX	Store accumulator in memory
BUN	4XXX	CXXX	Branch unconditionally
BSA	5XXX	DXXX	Branch and save return address
INC	6XXX	EXXX	Increment

Address is written in hexadecimal notation.

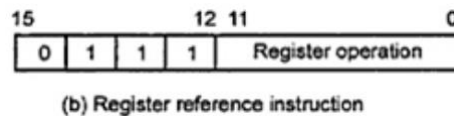
TABLE 5-4 Memory-Reference Instructions

Symbol	Operation decoder	Symbolic description
AND	D_0	$AC \leftarrow AC \wedge M[AR]$
ADD	D_1	$AC \leftarrow AC + M[AR], E \leftarrow C_{out}$
LDA	D_2	$AC \leftarrow M[AR]$
STA	D_3	$M[AR] \leftarrow AC$
BUN	D_4	$PC \leftarrow AR$
BSA	D_5	$M[AR] \leftarrow PC, PC \leftarrow AR + 1$
ISZ	D_6	$M[AR] \leftarrow M[AR] + 1,$ If $M[AR] + 1 = 0$ then $PC \leftarrow PC + 1$

AND instruction is executed when decoder output is D_0 ,and timing signal is T4.

2. Register Reference Instruction

These instructions are mainly used to perform operations on registers. Operands are not required here.

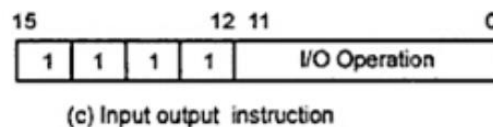


From bit 0 to bit 11 specifies register operation. Here opcode value always remains 111, and the value of the indirect bit is always 0. (figure b).

Register reference instructions are executed when decoder output is D7 and timing signal outputs T3.

3. Input/Output Instruction

These instructions are mainly used to perform operations on input and output related registers.



From bit 0 to bit 11 specifies input output operation. Here opcode value always remains 111 and value of indirect bit is always 1.

The computer decides the instruction type by looking at bits 12 to 15. If bits 12 to 14 are not 111, it is a memory-reference instruction, and bit 15 is used to check the addressing mode. If bits 12 to 14 are 111, then bit 15 tells whether the instruction is register-reference (0) or input-output (1). In this case, bit 15 is not used as a mode bit.

Even though there are only 3 bits for the opcode (allowing 8 basic operations), more instructions can be added because the register-reference and input-output instructions use the other 12 bits to define their specific tasks. Altogether, the basic computer supports 25 different instructions.

The instructions are written using three-letter symbols (like ADD, LOA, HLT) which are easier for programmers to understand. Each instruction also has a hexadecimal code, which is a shorter way to write the 16-bit binary instruction using four digits. For example, the instruction ADD could be written as 9xxx, where 'xxx' stands for the 12-bit memory address.

TABLE 5-2 Basic Computer Instructions

Symbol	Hexadecimal code		Description
	I = 0	I = 1	
AND	0xxx	8xxx	AND memory word to AC
ADD	1xxx	9xxx	Add memory word to AC
LDA	2xxx	Axxx	Load memory word to AC
STA	3xxx	Bxxx	Store content of AC in memory
BUN	4xxx	Cxxx	Branch unconditionally
BSA	5xxx	Dxxx	Branch and save return address
ISZ	6xxx	Exxx	Increment and skip if zero
CLA	7800		Clear AC
CLE	7400		Clear E
CMA	7200		Complement AC
CME	7100		Complement E
CIR	7080		Circulate right AC and E
CIL	7040		Circulate left AC and E
INC	7020		Increment AC
SPA	7010		Skip next instruction if AC positive
SNA	7008		Skip next instruction if AC negative
SZA	7004		Skip next instruction if AC zero
SZE	7002		Skip next instruction if E is 0
HLT	7001		Halt computer
INP	F800		Input character to AC
OUT	F400		Output character from AC
SKI	F200		Skip on input flag
SKO	F100		Skip on output flag
ION	F080		Interrupt on
IOF	F040		Interrupt off

For memory-reference instructions, the last bit (I) affects the hexadecimal value. If I = 0, the last hex digit is between 0 and 6. If I = 1, it ranges from 8 to E.

Register-reference instructions always begin with 0111 in binary, which equals 7 in hexadecimal. The rest of the instruction (12 bits) forms the rest of the code.

Similarly, input-output instructions begin with 1111 in the last four bits, which is equal to hexadecimal F. These codes make it easier for programmers to write and understand the computer's instructions.

When we type a character on the keyboard, it gets sent to an input register (INPR). There's a flag (FGI) that shows if a character is present (FGI=1) or not (FGI=0). The character then moves from the input register to the accumulator. From there, it goes to the output register, which has its own flag (FGO). FGO starts at 1, but switches to 0 when a character is in the output register. Finally, the receiver interface takes the character from the output register and sends it to the monitor. That's how the transmission works!

Instruction Set Completeness

A set of instructions is said to be complete if the computer includes a sufficient number of instructions in each of the following categories:

- Arithmetic, logical and shift instructions
- A set of instructions for moving information to and from memory and processor registers.
- Instructions which control the program together with instructions that check status conditions.
- Input and Output instructions

Timing and Control in the Basic Computer

In a basic computer, the timing of all operations is controlled by a master clock generator. This clock sends regular pulses to all the registers and flip-flops in the system, including those in the control unit. However, a register will only respond to a clock pulse if it receives a control signal that enables it. These control signals are generated by the control unit and are responsible for tasks like choosing inputs for the common bus and controlling operations in the accumulator.

Two Types of Control Units

There are two main ways to organize the control unit:

1. **Hardwired Control** – This uses physical components like gates and flip-flops to create control logic. It works quickly but is difficult to modify because changes require rewiring the system.
2. **Microprogrammed Control** – In this method, control instructions are stored in memory. You can change how the computer operates by simply updating this program, making it more flexible.

Components of the Control Unit

The control unit has:

- Two decoders
- A sequence counter
- Several control logic gates

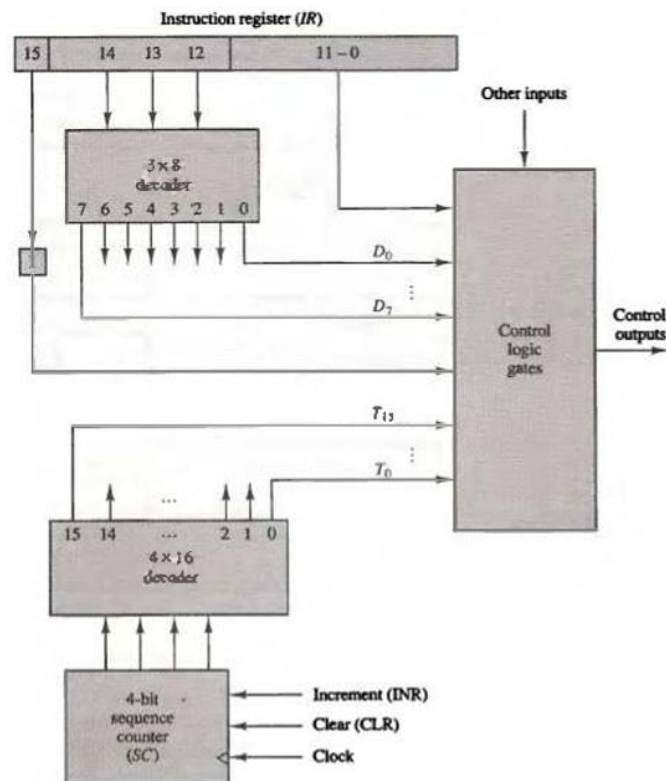


Figure 5-6 Control unit of basic computer.

When an instruction is read from memory, it is placed into the Instruction Register (IR). This register holds:

- The I bit (bit 15)
- The operation code (bits 12–14)
- Bits 0 to 11, which are typically used for addressing

The operation code is sent to a 3x8 decoder, which activates one of eight outputs labeled D0 to D7, depending on the code. Bit 15 is sent to a flip-flop named I, and bits 0 to 11 go to the control logic gates.

The Sequence Counter (SC) is a 4-bit counter that counts from 0 to 15. It controls a decoder that produces 16 different timing signals, labeled T0 to T15. These signals determine the exact timing of operations in the computer.

How the Sequence Counter Works

The sequence counter usually increases by 1 with each clock pulse, producing a series of timing signals. But sometimes, it gets reset to 0 to restart the sequence.

For example, if operation D3 is active at time T4, the counter will be reset. This is written as:

D3T4: $SC \leftarrow 0$

This means that when signal D3 and timing T4 are both active, the counter is cleared.

The timing diagram shows how these signals line up with the clock. Each timing signal (T0, T1, T2, etc.) is active for one clock cycle. Registers only respond if they are enabled during the timing signal and when the clock transitions positively (rises). If the counter isn't reset, the timing sequence continues up to T15 and then loops back to T0.

Memory Access and Timing

Reading or writing to memory is triggered by the rising edge of a timing signal. In our example, it's assumed that a memory operation finishes before the next clock cycle begins. This means that when the clock transitions again, the result of the memory operation is ready to be loaded into a register. In some computers, memory access takes longer than the processor's clock cycle, so wait cycles are needed. But for simplicity, the basic computer assumes no waiting is required.

Timing Example

To really understand how this works, you need to look at how clock transitions and timing signals work together.

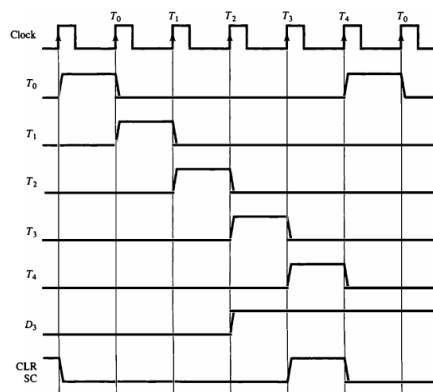


Figure 5-7 Example of control timing signals.

For example:

T0: $AR \leftarrow PC$

This means at timing signal T0, the content of the Program Counter (PC) is copied into the Address Register (AR). T0 stays active for one full clock cycle. During this time:

- PC places its value onto the bus
- AR's load signal is enabled

The actual transfer happens at the end of the clock cycle, when the clock makes a rising transition. That same clock transition also increases the sequence counter from 0000 to 0001, making T₁ active in the next cycle.

What is the Instruction Cycle?

A program residing in the memory unit of the computer consists of a sequence of instructions. The program is executed in the computer by going through a cycle for each instruction. Each instruction cycle in turn is subdivided into a sequence of sub cycles or phases.

In the basic computer each instruction cycle consists of the following phases:

1. Fetch an instruction from memory.
2. Decode the instruction.
3. Read the effective address from memory if the instruction has an indirect address.
4. Execute the instruction.

Upon the completion of step 4, the control goes back to step 1 to fetch, decode, and execute the next instruction. This process continues indefinitely unless a HALT instruction is encountered.

Fetch and Decode

The process starts with the Program Counter (PC), which holds the address of the first instruction in the program. The Sequence Counter (SC) is cleared to 0, generating the first timing signal, T₀. After each clock pulse, SC increases by 1, producing the sequence of timing signals: T₀, T₁, T₂, and so on.

$$\begin{aligned} T_0: & AR \leftarrow PC \\ T_1: & IR \leftarrow M[AR], \quad PC \leftarrow PC + 1 \\ T_2: & D_0, \dots, D_7 \leftarrow \text{Decode } IR(12-14), \quad AR \leftarrow IR(0-11), \quad I \leftarrow IR(15) \end{aligned}$$

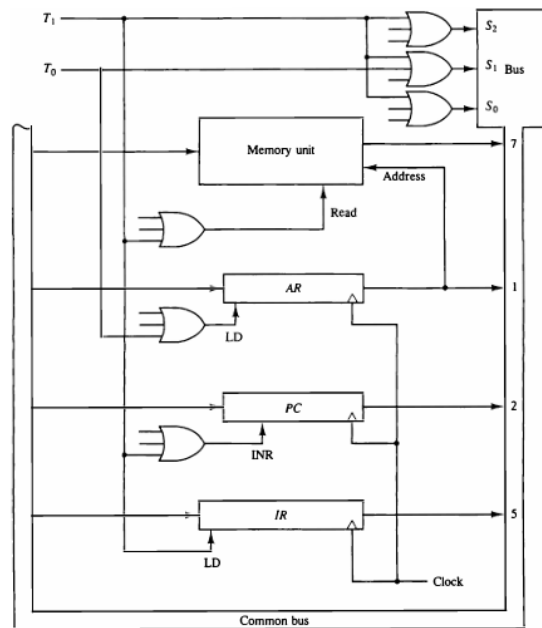


Figure 5-8 Register transfers for the fetch phase.

At T_0 , the contents of the PC are transferred to the Address Register (AR). This is important because only AR is directly connected to the memory's address lines. To achieve this using the bus system, the following happens:

- The value of PC is placed on the bus by setting the bus selection inputs $S_2 S_1 S_0 = 010$.
- AR is set to load from the bus by enabling its LD (load) signal.
- On the next clock pulse, AR receives the value from PC.

At T_1 , the instruction stored in memory at the address held in AR is fetched and placed in the Instruction Register (IR). Also, the PC is incremented to point to the next instruction. Here's what happens in this step:

- The memory read signal is enabled.
- The contents of memory at AR are placed on the bus using $S_2 S_1 S_0 = 111$.
- IR is set to load from the bus by enabling the LD signal of IR.
- PC is incremented by enabling its INR (increment) signal.
- On the next clock pulse, IR is loaded with the instruction and PC moves to the next address.

At T_2 , the fetched instruction in IR is decoded:

- The operation code (bits 12 to 14 of IR) is decoded into signals D_0 to D_7 .

- The address part of the instruction (bits 0 to 11) is transferred to AR.
- The indirect bit (bit 15 of IR) is transferred to a flip-flop named I.

This sequence — T_0 , T_1 , and T_2 — sets up the processor to understand and prepare for executing the instruction. Each of these steps happens one after the other, synchronized with the system clock through SC.

Determine the Type of Instruction

After decoding the instruction, the timing signal T_3 becomes active. During this time, the control unit figures out what type of instruction was just fetched from memory.

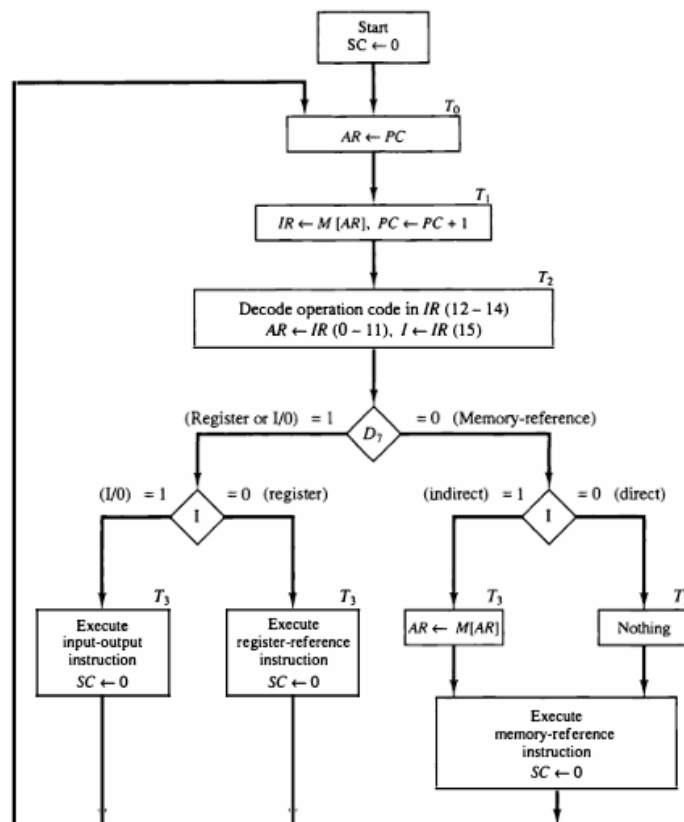


Figure 5-9 Flowchart for instruction cycle (initial configuration).

There are three types of instructions in the basic computer: memory-reference, register-reference, and input-output instructions. The type of instruction is identified using the decoder output D_7 .

If $D_7 = 1$, it means the instruction is either a register-reference or an input-output instruction. If $D_7 = 0$, then it is a memory-reference instruction.

Next, the control unit checks the indirect bit, which is stored in flip-flop I. If $D_7 = 0$ and $I = 1$, it means this is a memory-reference instruction that uses indirect addressing. In this case, the computer needs to go to memory again to get the effective address. The value in the Address Register (AR) is used to read from memory, and the result replaces the old address in AR. This is written as $AR \leftarrow M[AR]$.

If the instruction is direct addressing (i.e., $I = 0$), then nothing extra needs to be done, since AR already holds the correct address.

If the instruction is register-reference or input-output, it is executed immediately at T_3 . After this, the system resets and goes back to fetch the next instruction.

But if it's a memory-reference instruction, the system moves to the next step (T_4) to execute it.

The system either increases the step counter (SC) or resets it to 0, depending on what needs to be done next.

Register-Reference Instructions

Register-reference instructions are recognized by the control when $D_7 = 1$ and $I = 0$. These instructions use bits 0 through 11 of the instruction code to specify one of 12 instructions. These 12 bits are available in IR (0-11). They were also transferred to AR during time T_2 .

Each of these 12 bits can represent a different instruction. At time T_3 , the control unit checks which bit is active to decide what action to perform. This step is called r , which stands for when D_7 is 1, I is 0, and T_3 is active. ($D_7I'T_3$). The control function is chosen based on one specific bit in IR from bit 0 to 11. We call each bit B_i , where i is the bit number. So, each control function can be written as rB_i .

For example, the instruction CLA has the hexadecimal code 7800 (see Table 5-2), which gives the binary equivalent 0111 1000 0000 0000. The first bit is a zero and is equivalent to I' . The next three bits constitute the operation code and are recognized from decoder output D_7 . Bit 11 in IR is 1 and is recognized from B_{11} . The control function that initiates the microoperation for this instruction is $D_7I'T_3B_{11} = rB_{11}$.

Once a register-reference instruction is executed at T3, the sequence counter (SC) is cleared back to 0. This means the computer is ready to fetch the next instruction starting again from T0.

Some of these instructions do operations like clearing or flipping the AC or E register or shifting and increasing their values. Others are skip instructions, which tell the computer to skip the next step if a condition is true. For example, if AC is zero, negative, or positive.

There's also a special instruction called HLT (Halt). The HLT instruction clears a start-stop flip-flop S and stops the sequence counter from counting. To restore the operation of the computer, the start-stop flip-flop must be set manually.

TABLE 5-3 Execution of Register-Reference Instructions

$D_7I'T_3 = r$ (common to all register-reference instructions) $IR(i) = B_i$ [bit in $IR(0-11)$ that specifies the operation]		
	$r:$ $SC \leftarrow 0$	Clear SC
CLA	$rB_{11}:$ $AC \leftarrow 0$	Clear AC
CLE	$rB_{10}:$ $E \leftarrow 0$	Clear E
CMA	$rB_9:$ $AC \leftarrow \overline{AC}$	Complement AC
CME	$rB_8:$ $E \leftarrow \overline{E}$	Complement E
CIR	$rB_7:$ $AC \leftarrow \text{shr } AC, AC(15) \leftarrow E, E \leftarrow AC(0)$	Circulate right
CIL	$rB_6:$ $AC \leftarrow \text{shl } AC, AC(0) \leftarrow E, E \leftarrow AC(15)$	Circulate left
INC	$rB_5:$ $AC \leftarrow AC + 1$	Increment AC
SPA	$rB_4:$ If $(AC(15) = 0)$ then $(PC \leftarrow PC + 1)$	Skip if positive
SNA	$rB_3:$ If $(AC(15) = 1)$ then $(PC \leftarrow PC + 1)$	Skip if negative
SZA	$rB_2:$ If $(AC = 0)$ then $PC \leftarrow PC + 1$	Skip if AC zero
SZE	$rB_1:$ If $(E = 0)$ then $(PC \leftarrow PC + 1)$	Skip if E zero
HLT	$rB_0:$ $S \leftarrow 0$ (S is a start-stop flip-flop)	Halt computer

Memory Reference Instruction

There are seven memory-reference instructions, and they are identified by specific outputs (D0 to D6) from the instruction decoder. The address that the instruction needs to use (called the effective address) is stored in the Address Register (AR). This happens either during timing step T2 (if the instruction uses a direct address) or during T3 (if the instruction uses an indirect address).

The actual execution of a memory-reference instruction begins at time T4. Each instruction involves multiple micro-operations because the computer cannot perform logic or calculations directly on data stored in memory. Instead, the data must first be read into a register (like AC) where operations can be performed.

TABLE 5-4 Memory-Reference Instructions

Symbol	Operation decoder	Symbolic description
AND	D_0	$AC \leftarrow AC \wedge M[AR]$
ADD	D_1	$AC \leftarrow AC + M[AR], E \leftarrow C_{out}$
LDA	D_2	$AC \leftarrow M[AR]$
STA	D_3	$M[AR] \leftarrow AC$
BUN	D_4	$PC \leftarrow AR$
BSA	D_5	$M[AR] \leftarrow PC, PC \leftarrow AR + 1$
ISZ	D_6	$M[AR] \leftarrow M[AR] + 1,$ If $M[AR] + 1 = 0$ then $PC \leftarrow PC + 1$

1. AND to AC: Performs bitwise AND between AC and memory content at AR, stores result in AC.

Microoperations:

- $D_0T_4: DR \leftarrow M[AR]$
- $D_0T_5: AC \leftarrow AC \wedge DR, SC \leftarrow 0$

2. ADD to AC: Adds memory value at AR to AC; stores result in AC, and carry to E.

Microoperations:

- $D_1T_4: DR \leftarrow M[AR]$
- $D_1T_5: AC \leftarrow AC + DR, E \leftarrow C_{out}, SC \leftarrow 0$

3. LDA (Load to AC): Loads memory content at AR into AC.

Microoperations:

- $D_2T_4: DR \leftarrow M[AR]$
- $D_2T_5: AC \leftarrow DR, SC \leftarrow 0$

4. STA (Store AC): Stores AC content into memory at AR.

Microoperation:

- $D_3T_4: M[AR] \leftarrow AC, SC \leftarrow 0$

5. BUN (Branch Unconditionally): Transfers control to the instruction at memory address in AR.

Microoperation:

- $D_4T_4: PC \leftarrow AR, SC \leftarrow 0$

6. BSA (Branch and Save Return Address): Stores return address (PC) in memory, jumps to subroutine.

Microoperations:

- $D_5T_4: M[AR] \leftarrow PC$
- $D_5T_5: AR \leftarrow AR + 1, PC \leftarrow AR, SC \leftarrow 0$

7. ISZ (Increment and Skip if Zero): Increments memory value at AR; if result is 0, skips next instruction.

Microoperations:

- $D_6T_4: DR \leftarrow M[AR]$
- $D_6T_5: DR \leftarrow DR + 1$
- $D_6T_6: M[AR] \leftarrow DR, \text{ if } DR = 0 \text{ then } PC \leftarrow PC + 1, SC \leftarrow 0$

Input-Output Instructions

Input-output instructions handle data transfer between the accumulator (AC) and I/O devices, check flag bits, and control interrupts. These instructions are identified by the opcode 1111, with $D_7 = 1$ and $I = 1$. They are executed during timing signal T3, and the sequence counter (SC) is reset afterward.

The INP instruction moves data from the input register (INPR) to the lower 8 bits of AC and clears the input flag. The OUT instruction sends the lower 8 bits of AC to the output register (OUTR) and clears the output flag.

Some instructions check flag bits. If the flag is 1, the next instruction (usually a branch) is skipped; otherwise, it is executed. This helps wait until the device is ready.

The last two instructions enable or disable the interrupt system using a control flip-flop called IEN.

TABLE 5-5 Input-Output Instructions

$D_7IT_3 = p$ (common to all input-output instructions)		
$IR(i) = B_i$ [bit in $IR(6-11)$ that specifies the instruction]		
	$p:$	$SC \leftarrow 0$
INP	$pB_{11}:$	$AC(0-7) \leftarrow INPR, FGI \leftarrow 0$
OUT	$pB_{10}:$	$OUTR \leftarrow AC(0-7), FGO \leftarrow 0$
SKI	$pB_9:$	If $(FGI = 1)$ then $(PC \leftarrow PC + 1)$
SKO	$pB_8:$	If $(FGO = 1)$ then $(PC \leftarrow PC + 1)$
ION	$pB_7:$	$IEN \leftarrow 1$
IOF	$pB_6:$	$IEN \leftarrow 0$

Common Bus System

The basic computer has eight registers, a memory unit and a control unit. Paths must be provided to transfer information from one register to another and between memory and registers. A more efficient scheme for transferring information in a system with many registers is to use a common bus. The connection of the registers and memory of the basic computer to a common bus system is shown below:

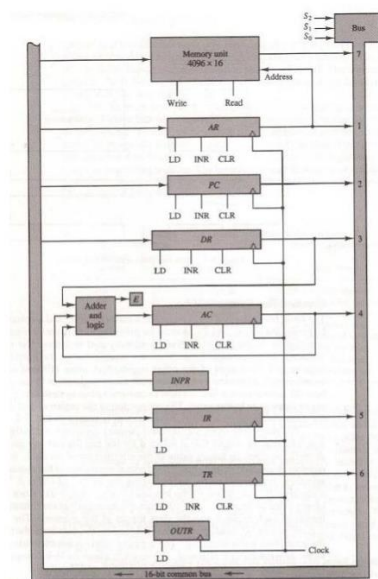


Fig. Basic Computer registers connected to a common bus

The outputs of seven registers and memory are connected to the common bus. The specific output that is selected for the bus lines at any given time is determined from the binary value of the selection variables S_2 , S_1 and S_0 .

The number along each output shows the decimal equivalents of the required binary selection. e.g. the number along the output of DR is 3. The 16-bit outputs of DR are placed on the bus lines when $S_2S_1S_0 = 011$.

The lines from the common bus are connected to the inputs of each register and the data inputs of the memory. The register whose LD (Load) input is enabled receives the data from the bus during the next clock pulse transition.

- The memory receives the contents of the bus when its write input is activated. The memory places its 16-bit output onto the bus when the read input is activated and $S_2S_1S_0 = 111$.

- Two registers AR and PC, have 12 bits each since they hold a memory address. When the contents of AR or PC are applied to the 16-bit common bus, the four most significant bits are set to 0's.
- Five registers have three control inputs: LD (Load), INR (increment) and CLR (clear).
- The input data and output data of the memory are connected to the common bus, but the memory address is connected to AR. Therefore, AR must always be used to specify a memory address.
- The 16 inputs of AC come from an adder and logic circuit. This circuit has three sets of inputs:
 - One set of 16-bit inputs come from the outputs of AC.
 - Another set of 17-bit inputs come from the data register DR.
 - The result of an addition is transferred to AC and the end carry-out of the addition is transferred to flip-flop E (extended AC bit).
 - A third set of 8-bit inputs come from the input register INPR.
- The content of any register can be applied onto the bus and an operation can be performed in the adder and logic circuit during the same clock cycle.

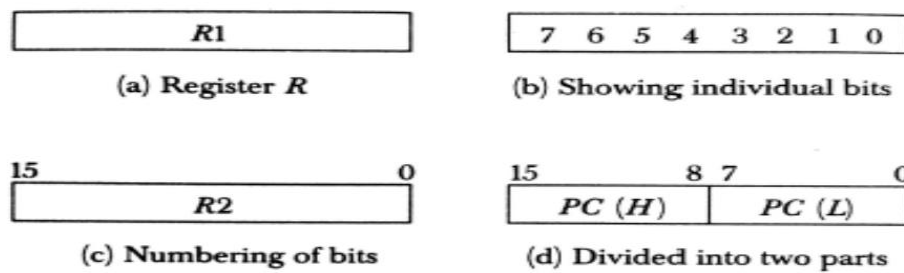
Register Transfer Language

Register Transfer Language (RTL) is a low-level language that is used to describe the functioning of a digital circuit and, more specifically, the transfer of information between registers. It provides how data moves from one register to the other and how data is processed within the digital system.

Registers:

- Computer registers are designated by upper case letters (and optionally followed by digits or letters) to denote the function of the register.
- For example, the register that holds an address for the memory unit is usually called a memory address register and is designated by the name MAR.
- Other designations for registers are PC (for program counter), IR (for instruction register, and R1 (for processor register).
- The individual flip-flops in an n-bit register are numbered in sequence from 0 through n-1, starting from 0 in the rightmost position and increasing the numbers toward the left.

Figure 4-1 Block diagram of register.



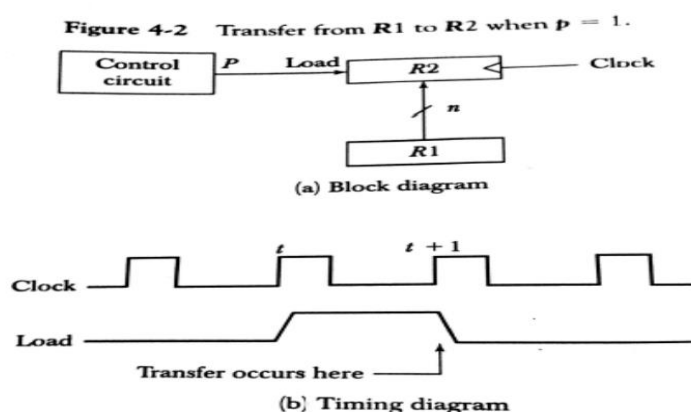
- The most common way to represent a register is by a rectangular box with the name of the register inside.
- The individual bits can be distinguished as in (b).
- The numbering of bits in a 16-bit register can be marked on top of the box as shown in (c).
- 16-bit register is partitioned into two parts in (d). Bits 0 through 7 are assigned the symbol L (for low byte) and bits 8 through 15 are assigned the symbol H (for high byte).
- The name of the 16-bit register is PC. The symbol PC (0-7) or PC (L) refers to the low-order byte and PC (8-15) or PC (H) to the high-order byte.

Register Transfer:

- Information transfer from one register to another is designated in symbolic form by means of a replacement operator.
- The statement $R2 \leftarrow R1$ denotes a transfer of the content of register R1 into register R2.
- It designates a replacement of the content of R2 by the content of R1.
- By definition, the content of the source register R 1 does not change after the transfer.
- If we want the transfer to occur only under a predetermined control condition, then it can be shown by an if-then statement.

if (P=1) then $R2 \leftarrow R1$

- P is the control signal generated by a control section.
- We can separate the control variables from the register transfer operation by specifying a Control Function.
- Control function is a Boolean variable that is equal to 0 or 1.
- control function is included in the statement as **P: $R2 \leftarrow R1$**
- Control condition is terminated by a colon implies transfer operation be executed by the hardware only if P=1.
- Every statement written in a register transfer notation implies a hardware construction for implementing the transfer.
- Figure 4-2 shows the block diagram that depicts the transfer from R1 to R2.



The basic symbols of the register transfer notation are listed in below table

Symbol	Description	Examples
Letters(and numerals)	Denotes a register	MAR, R2
Parentheses ()	Denotes a part of a register	R2(0-7), R2(L)
Arrow $\leftarrow$	Denotes transfer of information	$R2 \leftarrow R1$
Comma ,	Separates two microoperations	$R2 \leftarrow R1, R1 \leftarrow R2$

- A comma is used to separate two or more operations that are executed at the same time.
- The statement $T : R2 \leftarrow R1, R1 \leftarrow R2$ (exchange operation) denotes an operation that exchanges the contents of two registers during one common clock pulse provided that $T=1$.

Common Bus System:

Bus and Memory Transfers:

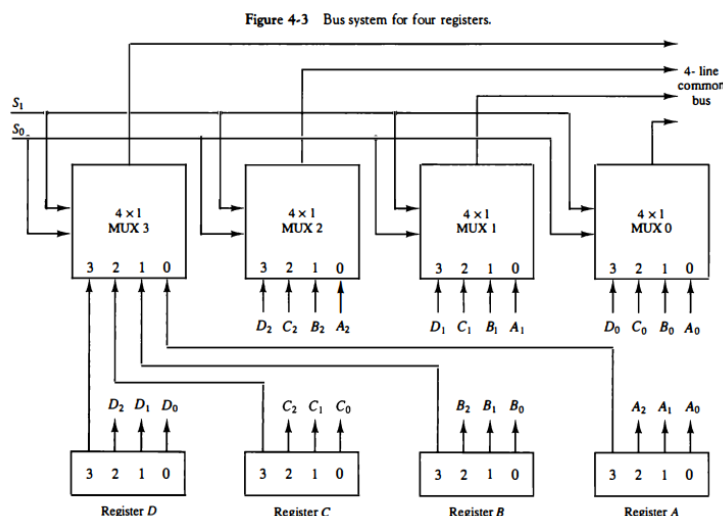
A typical digital computer has many registers, and paths must be provided to transfer information from one register to another.

The number of wires will be excessive if separate lines are used between each register and all other registers in the system.

A more efficient scheme for transferring information between registers in a multiple-register configuration is a **common bus system**.

A bus structure consists of a set of common lines, one for each bit of a register, determine which register is selected by the bus during each particular register transfer. One way of constructing a common bus system is with multiplexers.

The multiplexers select the source register whose binary information is then placed on the bus. The construction of a bus system for four registers is shown in Fig. 4-3.



Each register has four bits, numbered 0 through 3.

The bus consists of four 4 x 1 multiplexers each having four data inputs, 0 through 3, and two selection inputs, S_1 and S_0 .

In order not to complicate the diagram with 16 lines crossing each other, we use labels to show the connections from the outputs of the registers to the inputs of the multiplexers. For example, output 1 of register A is connected to input 0 of MUX 1 because this input is labeled A_1 .

The diagram shows that the bits in the same significant position in each register are connected to the data inputs of one multiplexer to form one line of the bus. Thus MUX 0 multiplexes the four 0 bits of the registers, MUX 1 multiplexes the four 1 bits of the registers, and similarly for the other two bits.

The two selection lines S_1 and S_0 are connected to the selection inputs of all four multiplexers. The selection lines choose the four bits of one register and transfer them into the four-line common bus.

When $S_1S_0 = 00$, the 0 data inputs of all four multiplexers are selected and applied to the outputs that form the bus. This causes the bus lines to receive the content of register A since the outputs of this register are connected to the 0 data inputs of the multiplexers.

Similarly, register B is selected if $S_1S_0 = 01$, and so on.

Table 4-2 shows the register that is selected by the bus for each of the four possible binary value of the selection lines.

TABLE 4-2 Function Table for Bus of Fig. 4-3

S_1	S_0	Register selected
0	0	A
0	1	B
1	0	C
1	1	D

In general, a bus system will multiplex k registers of n bits each to produce an n -line common bus. The number of multiplexers needed to construct the bus is equal to n , the number of bits in each register. The size of each multiplexer must be $k \times 1$ since it multiplexes k data lines. For example, a common bus for eight registers of 16 bits each requires 16 multiplexers, one for each line in the bus. Each multiplexer must have eight data input lines and three selection lines to multiplex one significant bit in the eight registers.

The transfer of information from a bus into one of many destination registers can be accomplished by connecting the bus lines to the inputs of all destination registers and activating the load control of the particular destination register selected. The symbolic statement for a bus transfer may mention the bus or its presence may be implied in the statement.

When the bus is included in the statement, the register transfer is symbolized as follows:

$$\text{BUS} \leftarrow C, R1 \leftarrow \text{BUS}$$

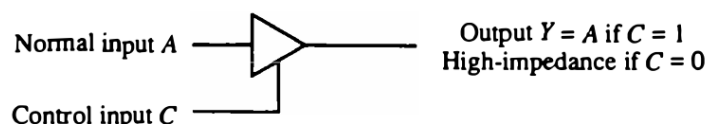
The content of register C is placed on the bus, and the content of the bus is loaded into register $R1$ by activating its load control input. If the bus is known to exist in the system, it may be convenient just to show the direct transfer.

$$R1 \leftarrow C$$

From this statement the designer knows which control signals must be activated to produce the transfer through the bus.

Three-State Bus Buffers:

- A bus system can be constructed with three-state gates instead of multiplexers.
- A three-state gate is a digital circuit that exhibits three states.
- Two of the states are signals equivalent to logic 1 and 0 as in a conventional gate.
- The third state is a high-impedance state.
- The high-impedance state behaves like an open circuit, which means that the output is disconnected and does not have logic significance.
- Because of this feature, a large number of three-state gate outputs can be connected with wires to form a common bus line without endangering loading effects.
- The graphic symbol of a three-state buffer gate is shown in Fig.



- It is distinguished from a normal buffer by having both a normal input and a control input.

- The control input determines the output state. When the control input is equal to 1, the output is enabled and the gate behaves like any conventional buffer, with the output equal to the normal input.
- When the control input is 0, the output is disabled and the gate goes to a high-impedance state, regardless of the value in the normal input.
- The construction of a bus system with three-state buffers is shown in Fig.

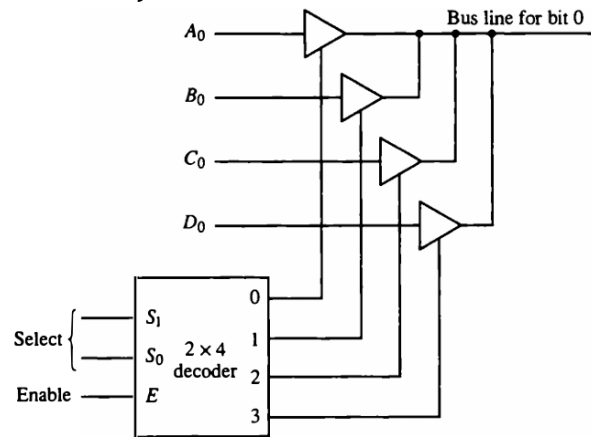


Figure 4-5 Bus line with three state-buffers.

- The outputs of four buffers are connected together to form a single bus line.
- The control inputs to the buffers determine which of the four normal inputs will communicate with the bus line.
- No more than one buffer may be in the active state at any given time. The connected buffers must be controlled so that only one three-state buffer has access to the bus line while all other buffers are maintained in a high impedance state.
- One way to ensure that no more than one control input is active at any given time is to use a decoder, as shown in the diagram.
- When the enable input of the decoder is 0, all of its outputs are 0, and the bus line is in a high-impedance state because all four buffers are disabled.
- When the enable input is active, one of the three-state buffers will be active, depending on the binary value in the select inputs of the decoder.

Memory Transfer:

- The transfer of information from a memory word to the outside environment is called a **read operation**.
- The transfer of new information to be stored into the memory is called a **write operation**.
- A memory word will be symbolized by the letter **M**.
- The particular memory word among the many available is selected by the memory address during the transfer.
- It is necessary to specify the address of **M** when writing memory transfer operations.
- This will be done by enclosing the address in square brackets following the letter **M**.
- Consider a memory unit that receives the address from a register, called the address register, symbolized by **AR**.
- The data are transferred to another register, called the data register, symbolized by **DR**.
- The read operation can be stated as follows:

Read: DR ← M [AR]

- This causes a transfer of information into DR from the memory word M selected by the address in **AR**.
- The write operation transfers the content of a data register to a memory word M selected by the address. Assume that the input data are in register R_1 and the address is in **AR**.
- The write operation can be stated as follows:

Write: $M[AR] \leftarrow R_1$

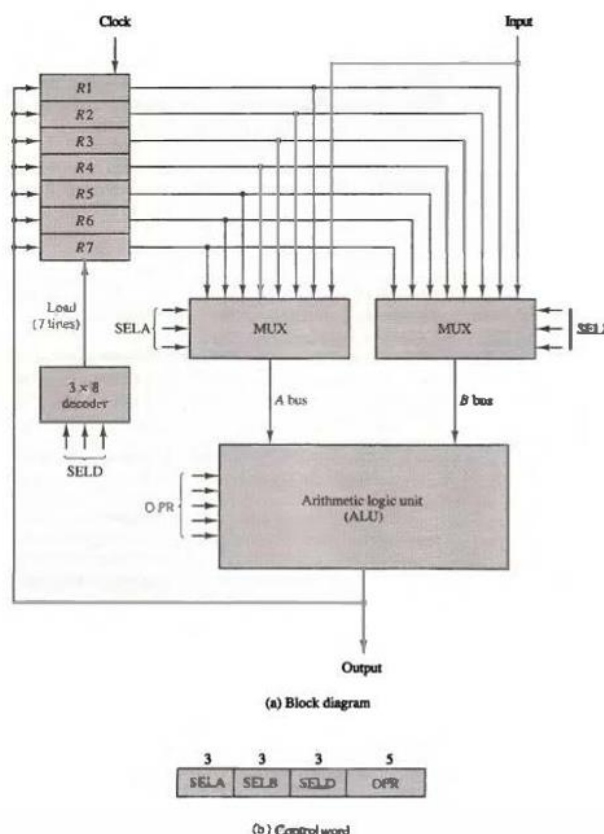
General Register Organization

In General Register Organization, the CPU uses a set of general-purpose registers to store data temporarily during program execution. These registers are small, high-speed storage locations within the CPU. Unlike accumulator-based architectures, where data is temporarily stored in a single accumulator, this organization uses multiple registers, each capable of storing different types of data such as integers, floating-point numbers, addresses, and control information.

The use of these general-purpose registers allows for faster data processing since accessing data from registers is significantly faster than fetching it from memory. Additionally, many CPU instructions are designed to operate directly on data stored in registers, which further accelerates computation.

The organization of general registers can vary between different CPU architectures and designs, impacting the CPU's performance and capabilities.

A bus organization for seven CPU registers is shown below:



The output of each register is connected to two multiplexers (MUX) to form the two buses A and B. The selection lines in each multiplexer select one register or the input data for the particular bus. These buses carry data to the ALU (Arithmetic Logic Unit). The ALU performs the selected operation (like addition or logic tasks) using the data from these buses.

The result of the operation can be:

- Sent out as output
- Sent back into the system so it can be stored in any of the registers

A decoder decides which register should store the result. The decoder activates one of the register load inputs, allowing the data to move from the output bus into it.

The control unit manages everything. It controls the flow of data by selecting which components (registers, buses, ALU, etc.) should be active at each moment. For example, to perform the operation

$$R1 \leftarrow R2 + R3$$

The following control signals are needed:

1. MUX A selector (SELA): to place the content of R2 into bus A.
2. MUX B selector (SELB): to place the content of R3 into bus B.
3. ALU operation selector (OPR): to provide the arithmetic addition $A + B$.
4. Decoder destination selector (SELD): to transfer the content of the output bus into R1.

These control signals are set at the beginning of the clock cycle. During the cycle, data flows from R2 and R3 through the MUXes and ALU, and the result appears on the output bus. At the next clock transition, the result is stored in R1.

Control word

The control word is determined by the sum of the binary selection inputs. The CPU control unit uses a 14-bit control word made up of four parts (or fields). Three fields contain three bits each, and one field has five bits, to control data movement and operations. The control word is as follows:



Here's how it's organized:

- SELA (3 bits): Chooses a register for Bus A (input to ALU)
- SELB (3 bits): Chooses a register for Bus B (input to ALU)
- SELD (3 bits): Chooses which register will store the result (via a decoder)
- OPR (5 bits): Selects the ALU operation (like add, subtract, AND, etc.)

These 14 bits together form a control word that tells the CPU exactly what micro-operation to perform.

The encoding for register selection is given in table.

Binary Code	SELA	SELB	SELD
000	Input	Input	None
001	R1	R1	R1
010	R2	R2	R2
011	R3	R3	R3
100	R4	R4	R4
101	R5	R5	R5
110	R6	R6	R6
111	R7	R7	R7

The 3-bit binary codes shown in the first column represent the values used in the SELA, SELB, and SELD fields of the control word. Each field selects a register based on the decimal equivalent of the 3-bit binary code.

- If SELA or SELB = 000, it means the external input is selected instead of a register.
- If SELD = 000, no register is selected to store the result, but the data from the output bus is still available externally.

The ALU performs both arithmetic and logic operations. Besides that, the CPU must also support shift operations. The shifter can be placed:

- At the input of the ALU for preshifting, or
- At the output of the ALU for postshifting

In some designs, shift operations are built into the ALU itself.

Function table for this ALU is listed in Table below:

TABLE 8-2 Encoding of ALU Operations

OPR Select	Operation	Symbol
00000	Transfer A	TSFA
00001	Increment A	INCA
00010	Add $A + B$	ADD
00101	Subtract $A - B$	SUB
00110	Decrement A	DECA
01000	AND A and B	AND
01010	OR A and B	OR
01100	XOR A and B	XOR
01110	Complement A	COMA
10000	Shift right A	SHRA
11000	Shift left A	SHLA

The OPR field of the control word has 5 bits, and each operation is identified by a symbolic name.

Examples of Microoperations:

A 14-bit control word is needed to define each microoperation in the CPU. This control word is made using selection fields that determine which registers and operation to use.

For example, take the microoperation:

$$R1 \leftarrow R2 - R3$$

This means:

- $R2$ is sent to ALU input A
- $R3$ is sent to ALU input B
- The result goes into $R1$
- The ALU performs subtraction ($A - B$)

To form the control word, we use the binary codes from Tables 8-1 and 8-2:

- SELA ($R2$) = 010
- SELB ($R3$) = 011
- SELD ($R1$) = 001
- OPR (SUB) = 00101

So, the control word becomes:

010 011 001 00101

This same method is used to form control words for other microoperations

Stack organization

What is a Stack?

A stack is a linear data structure that stores data in a sequential manner and follows LIFO. Data is added to or removed from one end, commonly referred to as the top of the stack. The primary operations on a stack are Push (adding an element) and Pop (removing an element). Stacks are often used for managing function calls, keeping track of expressions, and supporting algorithms such as depth-first search.

Stack organization

Stack organization refers to the way a computer system uses a stack for managing execution flow, storing temporary data, and performing arithmetic or logical operations. The stack is implemented as a portion of memory, with a stack pointer (SP) that tracks the current top of the stack.

The primary operations in a stack organization are:

- Push: Pushes an element to the top of the stack.
- Pop: Removes the topmost element from the stack.

In stack organization, when an operation is performed, the stack pointer is updated. This makes stack operations efficient and allows for orderly memory access.

Implementation of Stack

A stack can be implemented using two primary methods:

- Array-Based Implementation
- Linked List-Based Implementation

Types of Stack Organization

There are two primary types of stack organization in computer architecture:

1. Register Stack

In a register stack, memory words or registers are placed on top of each other. The address of the top element is stored in the stack pointer register. Each time an element is added (pushed), the stack pointer is incremented. When an element is removed (popped), the pointer is decremented.

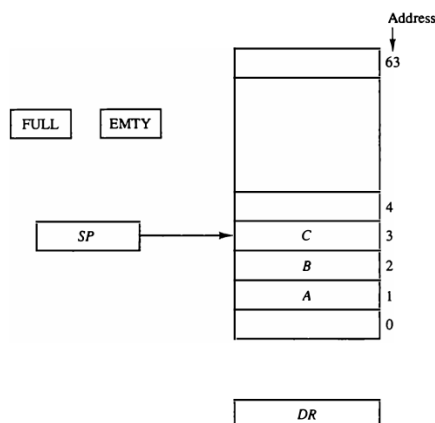


Figure 8-3 Block diagram of a 64-word stack.

Where,

- Stack Pointer (SP): Points to the top of the stack.
- Data Register (DR): Holds data being transferred.
- Full: The stack is at maximum capacity and cannot hold more data (stack overflow).
- Empty: The stack contains no data, and no items can be popped (stack underflow).

Let's consider a 64-word register stack. This stack is managed using a stack pointer (SP), which holds the binary address of the top element in the stack.

The three-element A, B, and C are in the stack.

The element C is at the top of the stack and the stack pointer holds the address of C that is 3. The top element is popped from the stack through reading memory word at address 3 and decrementing the stack pointer by 1. Then, B is at the top of the stack and the SP holds the address of B that is 2. It can insert a new word; the stack is pushed by incrementing the stack pointer by 1 and inserting a word in that incremented location.

- The stack can hold 64 words, so SP must represent values from 0 to 63
- Therefore, it uses 6 bits (because $2^6=64$)
- SP can go from 000000 (0) to 111111 (63)

⚠ Overflow & Underflow

- If SP = 111111 (63) and we increment it, it becomes 1000000 → only the lower 6 bits are kept, so SP becomes 000000 (0)
- If SP = 000000 (0) and we decrement it, it wraps around to 111111 (63)

♀ Status Flags: EMTY & FULL

Two status registers track the stack's condition:

- $FULL = 1 \rightarrow$ The stack is full (no more elements can be pushed)
- $EMPTY = 1 \rightarrow$ The stack is empty (no more elements to pop)

There's also a Data Register (DR) that temporarily stores the data being written to or read from the stack.

First, the SP is set to 0, EMPTY is set to 1, and FULL is set to 0. Now, as the stack is not full ($FULL = 0$), a new element is inserted using the push operation.

The push operation is executed as follows –

$SP \leftarrow SP + 1$	It can increment stack pointer
$K[SP] \leftarrow DR$	It can write element on top of the stack
If ($SP = 0$) then ($FULL \leftarrow 1$)	Check if stack is full
$EMPTY \leftarrow 0$	Mark the stack not empty

A new element is deleted from the stack if the stack is not empty (if $EMPTY = 0$). The pop operation includes the following sequence of micro-operations –

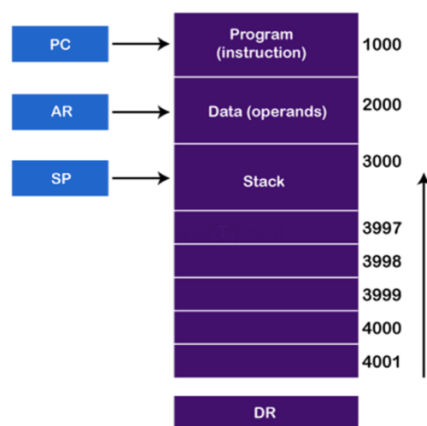
$DR \leftarrow K[SP]$	It can read an element from the top of the stack
$SP \leftarrow SP - 1$	It can decrement the stack pointer
If ($SP = 0$) then ($EMPTY \leftarrow 1$)	Check if stack is empty
$FULL \leftarrow 0$	Mark the stack not full

The top element from the stack is read and transfer to DR and thus the stack pointer is decremented. If the stack pointer reaches 0, then the stack is empty and 'EMPTY' is set to 1. This is the condition when the element in location 1 is read out and the SP is decremented by 1.

2.Memory Stack

A memory stack is created by reserving a portion of memory for the stack. The stack pointer (SP) points to the current top of the stack in memory. The stack typically grows downwards,

with new elements being pushed at lower memory addresses. The stack pointer register keeps track of the top of the stack, and elements are inserted or removed using this pointer.



Where,

- Program Counter (PC): Holds the address of the next instruction.
- Address Register (AR): Stores memory addresses.
- Stack Pointer (SP): Points to the top of the stack.
- Data Register (DR): Holds data being transferred.

The three registers are connected to a common address bus, and either one can provide an address for memory. PC is used during the fetch phase to read an instruction. AR is used during the execute phase to read an operand. SP is used to push or pop items into or from the stack.

As shown in Fig. the initial value of SP is 4001 and the stack grows with decreasing addresses. Thus, the first item stored in the stack is at address 4 000, the second item is stored at address 3999, and the last address that can be used for the stack is 3000. No provisions are available for stack limit checks.

We assume that the items in the stack communicate with a data register DR. A new item is inserted with the push operation as follows:

$$SP \leftarrow SP - 1$$

$$M[SP] \leftarrow DR$$

The stack pointer is decremented so that it points at the address of the next word. A memory write operation inserts the word from DR into the top of the stack.

A new item is deleted with a pop operation as follows:

$$DR \leftarrow M[SP]$$

$$SP \leftarrow SP + 1$$

The top item is read from the stack into DR. The stack pointer is then incremented to point at the next item in the stack.

Most computers do not provide hardware to check for stack overflow (full stack) or underflow (empty stack). The stack limits can be checked by using two processor registers: one to hold the upper limit (3000 in this case), and the other to hold the lower limit (4001 in this case). After a push operation, SP is compared with the upper-limit register and after a pop operation, SP is compared with the lower-limit register.

Instruction Formats

An instruction format refers to the layout or structure of a machine-level instruction, which consists of several fields that hold different types of information. Each field in an instruction has a specific role in telling the CPU what operation to perform, where the operands are located, and how to access them.

The typical components of an instruction format include:

- Opcode: The operation to be performed (e.g., addition, subtraction).
- Operands: The data or addresses involved in the operation.
- Addressing Modes: It defines how the operands are accessed (e.g., direct, indirect).

The instruction format directly influences how the CPU fetches, decodes, and executes instructions, making it a vital concept in computer organization.

A computer can have instructions of different lengths, each containing a varying number of addresses. The number of address fields in an instruction depends on how the computer's internal registers are designed. Most computers follow one of three main types of CPU organization:

single accumulator, general register, or stack-based.

In a single accumulator organization, all operations are carried out using a special register called the accumulator. Instructions typically have only one address field. For instance, the instruction ADD X means that the content at memory location X is added to the value already

in the accumulator, and the result is stored back in the accumulator. In symbolic form, this is written as $AC \leftarrow AC + M[X]$, where AC stands for the accumulator and $M[X]$ represents the value stored at address X.

In general register organization, instructions use two or three address fields. For example, ADD R1, X adds the value at memory location X to register R1 and stores the result in R1. This setup offers more flexibility and speed.

In stack organization, data is managed using a stack with PUSH and POP instructions. For example, PUSH X adds the value at address X to the top of the stack. Other operations work on the top two stack values, so they don't need explicit addresses.

Types of Instruction Formats

Based on the number of address fields used in the instruction, instruction formats are classified into zero, one, two, and three-address formats. Each has distinct characteristics and uses in different types of instruction formats in computer architecture.

1. Zero-Address Instructions

It operates without specifying any operands explicitly. Typically used in stack-based architectures, these instructions rely on the stack to store operands. The operations are performed on the top of the stack, and the results are also stored on the stack.



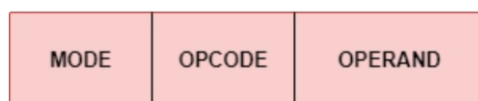
Example: Consider the below operations, which shows how $X = (A + B) * (C + D)$ expression will be written for a stack-organized computer.

In Postfix Notation, $X = (A + B) * (C + D)$ would be written as $AB+CD+*$.

PUSH	A	TOP = A
PUSH	B	TOP = B
ADD		TOP = A+B
PUSH	C	TOP = C
PUSH	D	TOP = D
ADD		TOP = C+D
MUL		TOP = (C+D)*(A+B)
POP	X	M[X] = TOP

2. One-Address Instructions

It specifies a single operand, typically held in an accumulator. The accumulator is an implicit register used in the operation. The result can either be stored in the accumulator or another location.



Example

The expression $X = (A + B) * (C + D)$ can be computed using one-address instructions with the accumulator (AC).

LOAD	A	AC = M[A]
ADD	B	AC = AC + M[B]
STORE	T	M[T] = AC
LOAD	C	AC = M[C]
ADD	D	AC = AC + M[D]
MUL	T	AC = AC * M[T]
STORE	X	M[X] = AC

All operations are done between the accumulator (AC) register and a memory operand.

M [A] is any memory location.

M[T] addresses a temporary memory location for storing the intermediate result.

This instruction format has only one operand field. This address field uses two special instructions to perform data transfer, namely:

- **LOAD:** This is used to transfer the data to the accumulator.
- **STORE:** This is used to move the data from the accumulator to the memory.

3. Two-Address Instructions

It allows for two operands to be specified. These operands can be registers or memory locations. The result can overwrite one of the source operands or can be stored in a separate location.

MODE	OPCODE	OPERAND 1	OPERAND 2
------	--------	-----------	-----------

Example: The expression $X = (A + B) * (C + D)$ can be calculated using two registers.

MOV	R1, A	$R1 = M[A]$
ADD	R1, B	$R1 = R1 + M[B]$
MOV	R2, C	$R2 = M[C]$
ADD	R2, D	$R2 = R2 + M[D]$
MUL	R1, R2	$R1 = R1 * R2$
MOV	X, R1	$M[X] = R1$

The MOV instruction transfers the operands to the memory from the processor registers. R1, R2 registers.

4. Three-Address Instructions

The format of a three address instruction requires three operand fields. These three fields can be either memory addresses or registers. This format allows for more complex operations and direct representation of expressions. It is most commonly used in high-level language compilers and optimizers.

MODE	OPCODE	OPERAND 1	OPERAND 2	OPERAND 3
------	--------	-----------	-----------	-----------

Example: The expression $X = (A + B) * (C + D)$ can be computed with three addresses.

ADD	R1, A, B	$R1 = M[A] + M[B]$
ADD	R2, C, D	$R2 = M[C] + M[D]$
MUL	X, R1, R2	$M[X] = R1 * R2$

Two processor registers, R1 and R2.

The symbol $M[A]$ denotes the operand at memory address symbolized by A. The operand1 and operand2 contain the data or address that the CPU will operate. Operand 3 contains the result's address.

Advantages of Instruction Formats

Here are the advantages of Instruction formats in computer organization:

- Allows predictable and efficient processing of instructions.
- Easier hardware design with consistent instruction structure.
- Reduces overhead and optimizes memory management.
- Simple structure speeds up instruction decoding.
- Simplifies code generation for better performance.
- Ensures software portability across systems.
- Accommodates a wide range of instructions

Disadvantages of Instruction Formats

Here are the disadvantages of instruction formats in computer organization:

- Balancing flexibility and efficiency can be difficult.
- Fixed format may restrict available instructions.
- Padding or unused bits reduces memory efficiency.
- Complex formats can slow down decoding.
- More complex formats may reduce speed.
- May not scale with new technologies.
- More bits can waste memory and bandwidth

Addressing Modes

What is Addressing Mode?

Addressing mode refers to the way in which the operand of an instruction is specified or accessed in a computer's memory.

It defines how the effective address of the operand is calculated and where it is located.

Different types of addressing modes exist, such as Immediate, Direct, Indirect, Register, and Indexed addressing.

Addressing modes provide flexibility in data access, allowing for more efficient memory usage and instruction execution.

Types of Addressing Modes

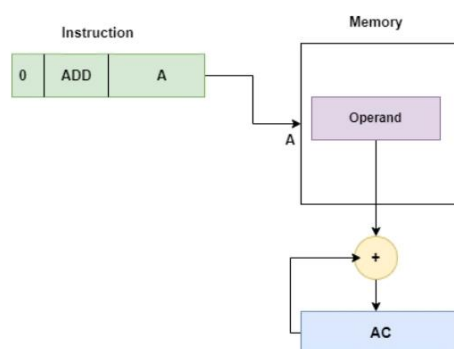
Addressing Modes in computer architecture are further divided into the following parts:

1. Register Direct Addressing Mode
2. Register Indirect Addressing Mode
3. Direct Addressing Mode
4. Indirect Addressing Mode
5. Indexed Addressing Mode etc...

1. Direct Addressing Mode:

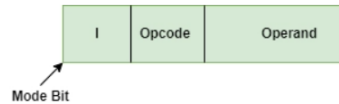
In this addressing mode, the address field of the instruction contains the effective address of the operand. Only one reference to memory is required to fetch the operand. It is also called as absolute addressing mode.

Example: ADD X will increment the value stored in the accumulator by the value stored at memory location X. $AC \leftarrow AC + [X]$

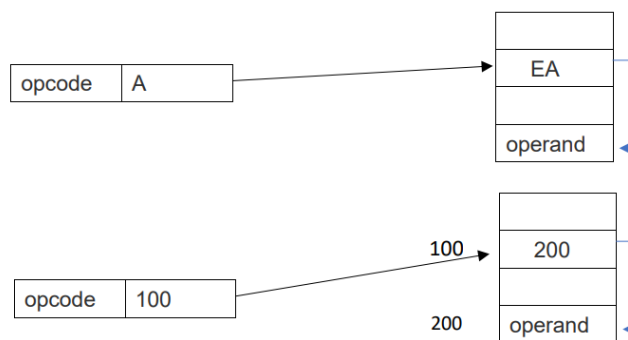


2. Indirect Addressing Mode:

In this addressing mode, the address field of the instruction specifies the address of memory location that contains the effective address of the operand. Two references to memory are required to fetch the operand. This addressing mode slows down the execution as it requires multiple memory lookups to find the operand.

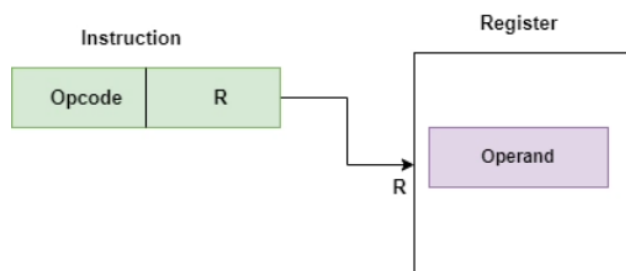


Example: ADD X will increment the value stored in the accumulator by the value stored at memory location specified by X. $AC \leftarrow AC + [[X]]$



3. Register Direct Addressing Mode:

In this addressing mode, the operand is contained in a register set. The address field of the instruction refers to a CPU register that contains the operand. No reference to memory is required to fetch the operand.



Example: ADD R will increment the value stored in the accumulator by the content of register R.

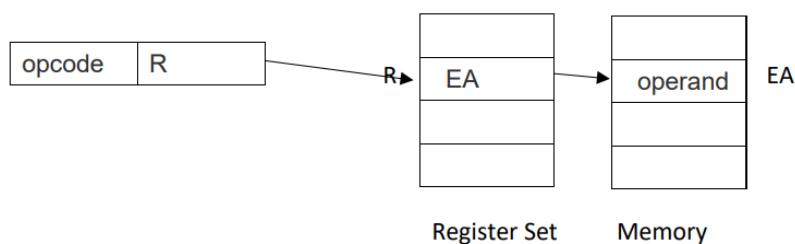
$$AC \leftarrow AC + [R]$$

- This addressing mode is similar to direct addressing mode.

- The only difference is address field of the instruction refers to a CPU register instead of main memory.

4.Register Indirect Addressing Mode:

In this addressing mode, the address field of the instruction refers to a CPU register that contains the effective address of the operand. Only one reference to memory is required to fetch the operand.



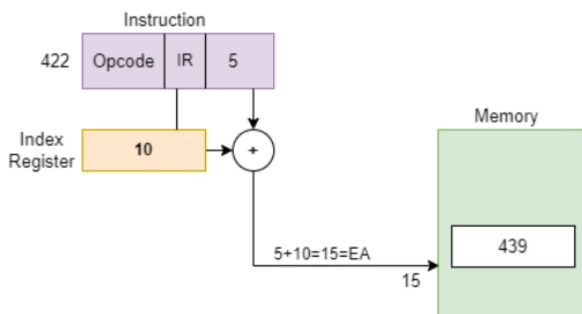
Example: ADD R will increment the value stored in the accumulator by the content of memory location specified in register R.

$$AC \leftarrow AC + [[R]]$$

- This addressing mode is similar to indirect addressing mode.
- The only difference is address field of the instruction refers to a CPU register.

5.Indexed Addressing Mode:

The index register's content is added to the instruction's address to obtain the effective address.
 $EA = \text{content of index register} + \text{Instruction address part}$



Reduced Instruction Set Computer (RISC)

RISC stands for Reduced Instruction Set Computer. In Reduced Instruction Set Computer (RISC) architecture, the instruction set of the computer is simplified to reduce the execution time. RISC has a small set of instructions, which generally include register-to-register operations.

Thus, data is stored in processor registers for computations, and results of the computations are transferred to the memory using store instructions. All operations are performed within the registers of the CPU. In RISC, all instructions have simple register addressing and hence use less number of addressing modes.

RISC uses relatively a simple instruction format and is easy to decode. Here, the instruction length can be fixed and aligned on word boundaries. The RISC processors can execute one instruction per clock cycle.

This is done using pipelining, which involves overlapping the fetch, decode, and execute phases of two or three instructions. As RISC takes relatively a large number of registers in the processor unit, it takes less time to execute its program when compared to CISC.

RISC Characteristics

The concept of RISC architecture involves an attempt to reduce execution time by simplifying the instruction set of the computer. The major characteristics of a RISC processor are:

1. Relatively few instructions
2. Relatively few addressing modes
3. Memory access limited to load and store instructions
4. All operations done within the registers of the CPU
5. Fixed-length, easily decoded instruction format
6. Single-cycle instruction execution
7. Hardwired rather than microprogrammed control

Complex Instruction Set Computer (CISC)

CISC stands for Complex Instruction Set Computer. It's a type of computer architecture where the processor has many instructions, often over 100 different commands. Some of these instructions are very powerful and complex, so a single instruction can do multiple tasks at

once — like reading from memory, performing a calculation, and saving the result — all in one step

The CISC architecture comprises a complex instruction set. A CISC processor has a variable-length instruction format. In this processor architecture, the instructions that require register operands can take only two bytes.

In a CISC processor architecture, the instructions which require two memory addresses can take five bytes to comprise the complete instruction code. Therefore, in a CISC processor, the execution of instructions may take a varying number of clock cycles. The CISC processor also provides direct manipulation of operands that are stored in the memory.

The primary objective of the CISC processor architecture is to support a single machine instruction for each statement that is written in a high-level programming language.

Example – An ADD instruction will use index addressing to specify one operand in memory and direct addressing to specify the second operand in memory. This instruction would use another memory location to store the result. Thus, this instruction would use three memory references for execution.

The following are the important characteristics of a CISC processor architecture?

1. CISC can have variable-length instruction formats.
2. It supports a set of a large number of instructions, typically from 100 to 250 instructions.
3. It has a large variety of addressing modes, typically from 5 to 20 different modes.
4. CISC has some instructions which perform specialized tasks and are used infrequently.

CISC architectures have a large, complex instruction set and a less efficient execution pipeline. This allows CISC processors to perform a wider range of tasks, but they are not as fast as RISC processors when executing instructions.

Difference between RISC and CISC

S.No.	RISC	CISC
-------	------	------

1.	It stands for Reduced Instruction Set Computer.	It stands for Complex Instruction Set Computer.
2.	It is a microprocessor architecture that uses small instruction set of uniform length.	This offers hundreds of instructions of different sizes to the users.
3.	These simple instructions are executed in one clock cycle.	This architecture has a set of special purpose circuits which help execute the instructions at a high speed.
4.	These chips are relatively simple to design.	These chips are complex to design.
5.	They are inexpensive.	They are relatively expensive.
6.	Examples of RISC chips include SPARC, POWER PC.	Examples of CISC include Intel architecture, AMD.
7.	It has less number of instructions.	It has more number of instructions.
8.	It has fixed-length encodings for instructions.	It has variable-length encodings of instructions.
9.	Simple addressing formats are supported.	The instructions interact with memory using complex addressing modes.
10.	It doesn't support arrays.	It has a large number of instructions. It supports arrays.
11.	It doesn't use condition codes.	Condition codes are used.
12.	Registers are used for procedure arguments and return addresses.	The stack is used for procedure arguments and return addresses.

Instruction Pipeline

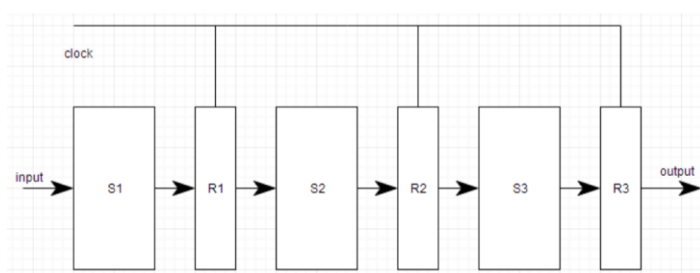
What is pipelining?

Pipelining is a technique where instructions are fetched, stored, and executed in a step-by-step flow. It helps the processor handle tasks in an organized and faster way. It's also called **pipeline processing**.

Pipelining is a technique where multiple instructions are overlapped during execution. Pipeline is divided into stages and these stages are connected with one another to form a pipe like structure. Instructions enter from one end and exit from another end.

Pipelining increases the overall instruction throughput.

In pipeline system, each segment consists of an input register followed by a combinational circuit. The register is used to hold data, and combinational circuit performs operations on it. The output of combinational circuit is applied to the input register of the next segment.



Types of Pipeline

It is divided into 2 categories:

1. Arithmetic Pipeline
2. Instruction Pipeline

Instruction Pipeline in Computer Architecture

Instruction Pipeline is a technique used in computer architecture to overlap the execution of multiple instructions, making the CPU faster and more efficient. An instruction pipeline allows various stages of multiple instructions to be processed simultaneously.

An instruction pipeline receives sequential instructions from memory while prior instructions are implemented in other portions. Pipeline processing can be seen in both the data and instruction streams.

Pipeline processing can happen not only in the data stream but also in the instruction stream. To perform tasks such as fetching, decoding and execution of instructions, most digital computers with complicated instructions would require an instruction pipeline.

In general, each and every instruction must be processed by the computer in the following order:

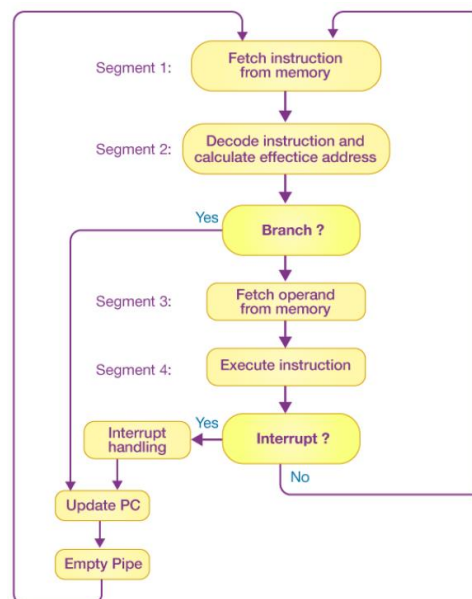
1. Fetching the instruction from memory

2. Decoding the obtained instruction
3. Calculating the effective address
4. Fetching the operands from the given memory
5. Execution of the instruction
6. Storing the result in a proper place

Each step is carried out in its own segment, and various segments may take different amounts of time to process the incoming data. Furthermore, there are occasions when multiple segments request memory access at the very same time, requiring one segment to wait unless and until the memory access of another is completed.

If the instruction cycle is separated into equal-length segments, the organization of an instruction pipeline will become much more efficient. A four-segment type of instruction pipeline refers to one of the most common instances of this style of organization.

A four-segment instruction pipeline unifies two or more distinct segments into a single unit. For example, the decoding of the instruction and the calculation of the effective address can be merged into a single segment.



A four-segment instruction pipeline is illustrated in the block diagram given above. The instructional cycle is divided into four parts:

Segment 1

The implementation of the instruction fetch segment can be done using the FIFO or first-in, first-out buffer.

Segment 2

In the second segment, the memory instruction is decoded, and the effective address is then determined in a separate arithmetic circuit.

Segment 3

In the third segment, some operands would be fetched from memory.

Segment 4

The instructions would finally be executed in the very last segment of a pipeline organization.

Memory Hierarchy

The memory unit is a key part of any digital computer. It stores both programs and data. While very simple computers might work without extra storage, most general-purpose computers need more memory than the main memory alone can provide.

In most cases, main memory cannot hold all the programs and data at once. Also, users keep collecting more software and data over time. Since not everything is needed all the time, it's better to use low-cost storage for the data that isn't being used at the moment. The memory that directly talks to the CPU is called main memory. Extra storage devices used for backup are called auxiliary memory.

Common auxiliary storage devices include magnetic disks and tapes. These store large files, system programs, and other backup data. Only the information currently needed by the CPU stays in the main memory. Other data is kept in auxiliary memory and moved to main memory when needed.

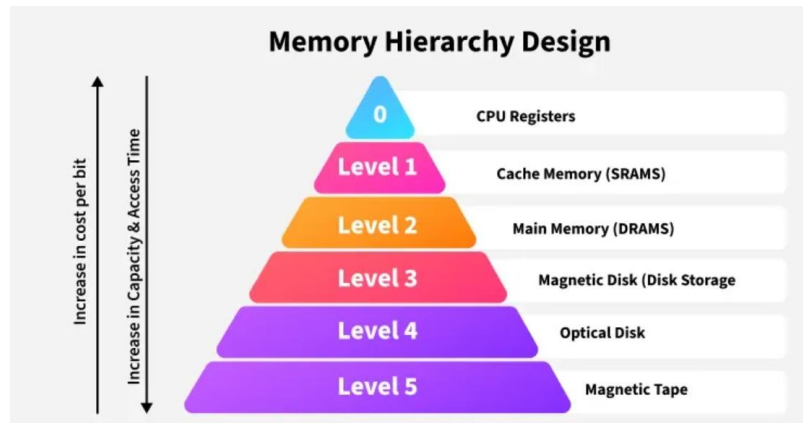
Computer memory is arranged in a layered system called the memory hierarchy.

At the bottom are slow but large storage devices like magnetic tapes. Then come magnetic disks. Main memory sits in the middle and talks directly to the CPU and auxiliary memory through input-output processors. When needed, data and programs are moved from auxiliary memory into main memory and vice versa.

To speed up processing, computers often use a very fast but small memory called cache. Cache stores the most frequently used data and instructions so the CPU can access them quickly. Since the CPU works faster than main memory, the cache helps close the speed gap and keeps the processor from waiting too long.

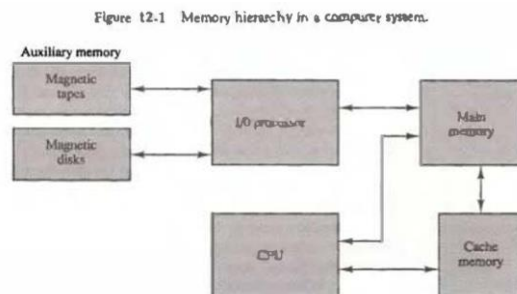
In the Computer System Design, Memory Hierarchy is an enhancement to organize the memory such that it can minimize the access time. The Memory Hierarchy was developed based on a program behavior known as locality of references.

The figure below clearly demonstrates the different levels of memory hierarchy:



Why Memory Hierarchy is Required in the System?

Memory Hierarchy helps in optimizing the memory available in the computer. There are multiple levels present in the memory, each one having a different size, different cost, etc. Some types of memory like cache, and main memory are faster as compared to other types of memory, but they are having a little less size and are also costly whereas some memory has a little higher storage value, but they are a little slower. Accessing of data is not similar in all types of memory, some have faster access whereas some have slower access.



This Memory Hierarchy Design is divided into 2 main types:

- 1. External Memory or Secondary Memory:** – Comprising of Magnetic Disk, Optical Disk, Magnetic Tape i.e. peripheral storage devices which are accessible by the processor via I/O Module.
- 2. Internal Memory or Primary Memory:** – Comprising of Main Memory, Cache Memory & CPU registers. This is directly accessible by the processor.

Characteristics of Memory Hierarchy

Capacity: It is the global volume of information the memory can store. As we move from top to bottom in the Hierarchy, the capacity increases.

Access Time: It is the time interval between the read/write request and the availability of the data. As we move from top to bottom in the Hierarchy, the access time increases.

Performance: The Memory Hierarchy design ensures that frequently accessed data is stored in faster memory to improve system performance.

Cost Per Bit: As we move from bottom to top in the Hierarchy, the cost per bit increases i.e. Internal Memory is costlier than External Memory.

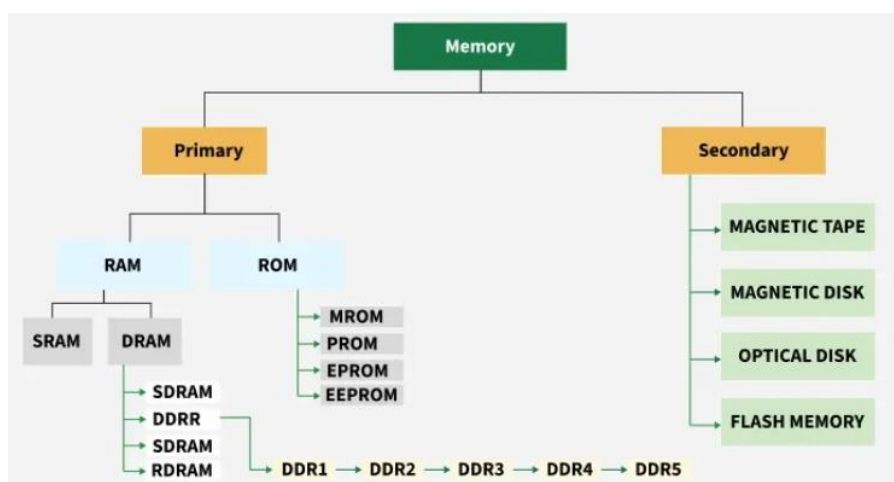
Main Memory

The main memory of a computer, also known as RAM (Random Access Memory), is a crucial component responsible for temporarily storing data and instructions that the CPU needs to access quickly during program execution. It serves as a workspace where the CPU can read and write data rapidly, enabling efficient processing of instructions.

Main memory is volatile, meaning its contents are lost when the computer is powered off. It consists of memory cells organized in a hierarchical structure, with each cell capable of storing a single binary digit (bit). These cells are grouped into bytes, which are the fundamental units of data storage in computers.

It is of two types:

- RAM (Random Access Memory)
- ROM (Read Only Memory)



RAM (Random Access Memory):

It is a volatile memory. Volatile memory stores information based on the power supply. If the power supply fails/ interrupted/stopped, all the data and information on this memory will be lost. RAM is used for booting up or starting the computer. It temporarily stores programs/data which has to be executed by the processor.

RAM is of two types:

- **S RAM (Static RAM):** Static RAM stores data using flip-flops with each flip flop storing 1 bit, and keeps the information as long as the computer has power. It has less access time and hence, it is faster.
- **D RAM (Dynamic RAM):** D RAM uses capacitors and transistors and stores the data as a charge on the capacitors. They contain thousands of memory cells. These charges slowly fade, so it needs refreshing of charge on capacitor after a few milliseconds. This memory is slower than S RAM.

ROM (Read Only Memory): It is a non-volatile memory. Non-volatile memory stores information even when there is a power supply failed/ interrupted/stopped. ROM is used to store information that is used to operate the system. As its name refers to read-only memory, we can only read the programs and data that are stored on it. It contains some electronic fuses that can be programmed for a piece of specific information. The information is stored in the ROM in binary format. It is also known as permanent memory. ROM is of four types:

- MROM
- PROM
- EPROM
- EEPROM

Memory Organization

The main memory technology is based on silicon integrated circuits. The main memory integrated circuits are divided into two major units:

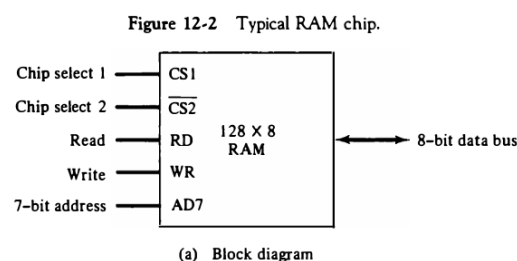
- RAM (Random Access Memory) integrated circuit chips.
- ROM (Read Only Memory) integrated circuit chips.

RAM Chip

A RAM (Random Access Memory) chip is designed to work closely with the CPU. It usually has control signals that make sure the chip is only active when needed. One important feature of RAM is the bidirectional data bus. This allows data to move in both directions—either from the memory to the CPU (when reading) or from the CPU to the memory (when writing).

A bidirectional bus can be constructed with three-state buffers. A three-state buffer output can be placed in one of three possible states: a signal equivalent to logic 1, a signal equivalent to logic 0, or a high impedance state. The logic 1 and 0 are normal digital signals. The high impedance state behaves like an open circuit, which means that the output does not carry a signal and has no logic significance.

The capacity of the memory is 128 words of eight bits (one byte) per word. This requires a 7-bit address and an 8-bit bidirectional data bus.



The read and write inputs control what action the memory should perform. The chip select (CS) inputs are used to turn on the chip only when the microprocessor needs it.

Having more than one chip select input makes it easier to manage which chip is active, especially when multiple memory chips are used together. Sometimes, the read and write signals are combined into a single line called R/W.

When the chip is selected, this line tells the chip whether to read data or write data based on whether the signal is 0 or 1.

The function table below shows how the RAM chip works.

CS1	$\overline{CS2}$	RD	WR	Memory function	State of data bus
0	0	x	x	Inhibit	High-impedance
0	1	x	x	Inhibit	High-impedance
1	0	0	0	Inhibit	High-impedance
1	0	0	1	Write	Input data to RAM
1	0	1	x	Read	Output data from RAM
1	1	x	x	Inhibit	High-impedance

(b) Function table

- The chip only operates when CS1 is 1 and $\overline{CS2}$ is 0. The bar over CS2 means it works when the value is 0.

- If these chip select inputs are not set correctly, or if the read (RD) or write (WR) inputs are not active, the chip stays off and its data bus goes into a high-impedance state (like being disconnected).
- When the chip is active ($CS1 = 1$ and $CS2 = 0$), it can either write or read data.
- If WR is active, the chip stores the data from the data bus into the memory location given by the address.
- If RD is active, the chip sends the stored data from that memory location to the data bus.

These RD and WR signals control how data moves in and out of the memory and also manage the bidirectional data bus.

ROM Chip

ROM (Read-Only Memory) chips work similarly on the outside but behave differently inside. However, since a ROM can only read, the data bus can only be in an output mode.

ROM chips can store more data than RAM chips of the same size because their internal cells take up less space. That's why the diagram shows a ROM with 512 bytes, while the RAM only has 128 bytes.

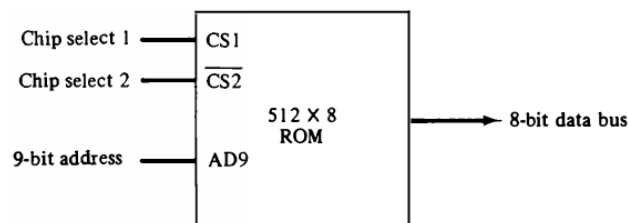


Figure 12-3 Typical ROM chip.

The ROM chip uses 9 address lines to access any one of the 512 stored bytes. The chip only works when $CS1$ is set to 1 and $\overline{CS2}$ is set to 0. If these conditions are not met, the data bus goes into a high-impedance state, meaning it doesn't send out any data.

Since ROM can only read, it doesn't need read or write control signals. When the chip is properly selected, the data from the chosen address is sent to the data bus automatically.

Memory Address Map

When designing a computer system, one of the key steps is to calculate the amount of memory needed for the specific application and divide it between RAM and ROM. Based on the required

size and the available RAM/ROM chips, the designer establishes how the memory will connect with the processor.

To assign memory locations properly, we use a memory address map. This map is a table (or diagram) that shows how each chip is assigned a specific range of addresses in the system.

TABLE 12-1 Memory Address Map for Microcomputer

Component	Hexadecimal address	Address bus									
		10	9	8	7	6	5	4	3	2	1
RAM 1	0000–007F	0	0	0	x	x	x	x	x	x	x
RAM 2	0080–00FF	0	0	1	x	x	x	x	x	x	x
RAM 3	0100–017F	0	1	0	x	x	x	x	x	x	x
RAM 4	0180–01FF	0	1	1	x	x	x	x	x	x	x
ROM	0200–03FF	1	x	x	x	x	x	x	x	x	x

OR

Component	Dec Address	Hex Address	Address Bus													
			16	15	14	13	12	11	10	9	8	7	6	5	4	3
RAM 1	000	0000	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	127	007F	0	0	0	0	0	0	0	0	0	1	1	1	1	1
RAM 2	128	0080	0	0	0	0	0	0	0	0	1	0	0	0	0	0
	255	00FF	0	0	0	0	0	0	0	0	1	1	1	1	1	1
RAM 3	256	0100	0	0	0	0	0	0	0	1	0	0	0	0	0	0
	383	017F	0	0	0	0	0	0	0	1	0	1	1	1	1	1
RAM 4	384	0180	0	0	0	0	0	0	0	1	1	0	0	0	0	0
	511	01FF	0	0	0	0	0	0	0	1	1	1	1	1	1	1
ROM	512	0200	0	0	0	0	0	0	1	0	0	0	0	0	0	0
	1023	03FF	0	0	0	0	0	0	1	1	1	1	1	1	1	1

Let's understand this with a practical example:

Suppose a system needs a total of 1024 bytes of memory, divided equally between RAM and ROM:

- RAM required = 512 bytes
- ROM required = 512 bytes

We are using:

- 128-byte RAM chips
- A single 512-byte ROM chip

Calculate number of chips needed:

Number of RAM chips = Total RAM / Size of each RAM chip

= 512 bytes / 128 bytes

= 4 RAM chips

Number of ROM chips = Total ROM / Size of each ROM chip

= 512 bytes / 512 bytes

= 1 ROM chip

Memory Map Table Components:

- The Component column in the table shows whether a RAM or ROM chip is used.
- The Decimal and Hexadecimal Address columns show the range of addresses assigned to each chip.
- The Address Bus Lines column shows which lines are used to access the memory.

Although the system has 16 address lines, in this example, only 10 lines are used. The remaining 6 lines are assumed to be 0.

Address Line Usage:

- Each 128-byte RAM chip requires 7 address lines (since $2^7 = 128$).
- The 512-byte ROM chip requires 9 address lines (since $2^9 = 512$).
- These address lines are always assigned to the lower-order bits of the address bus:
 - Lines 1 to 7 for RAM
 - Lines 1 to 9 for ROM

Distinguishing Between Multiple RAM Chips:

To select between the four RAM chips, we need 2 extra address lines (since $2^2 = 4$). We use address lines 8 and 9 for this purpose. These lines provide unique binary combinations (00, 01, 10, 11) to select each RAM chip.

Note: Any pair of unused address lines could be chosen here; lines 8 and 9 are just one valid option.

Distinguishing Between RAM and ROM:

To tell the system whether to access RAM or ROM, we use address line 10:

- When line 10 = 0, the system accesses RAM
- When line 10 = 1, the system accesses ROM

This address mapping ensures that the processor can uniquely select each memory chip and specific location within it, using the right combination of address lines and chip selection logic.

Hexadecimal addresses are based on how the address lines are grouped in sets of 4 bits. This grouping makes it easier to convert binary addresses into hex.

Memory Connection to CPU

In a computer system, RAM and ROM chips are connected to the CPU using two main channels: the data bus and the address bus.

The address bus is used to select memory locations. The lower bits of the address bus choose specific bytes within a chip, while the higher bits are used to select which chip to access using special control lines called chip select inputs.

This setup, shown in Fig. 12-4, provides a total memory of 512 bytes of RAM and 512 bytes of ROM. It follows the memory map given in Table 12-1. Each RAM chip uses seven low-order address lines to access one of its 128 bytes.

To choose which RAM chip to activate, the system uses address lines 8 and 9. These lines are connected to a 2x4 decoder, which sends signals to the chip select (CS1) inputs of the RAM chips. For example, if the values on lines 8 and 9 are 00, the first RAM chip is selected. If the values are 01, the second RAM chip is selected, and so on. The CPU also uses read (RD) and write (WR) signals to control whether data is being read from or written to the RAM.

Choosing between RAM and ROM is done using address line 10. If this line is 0, RAM is selected. If it is 1, the ROM is selected instead. The ROM also has a chip select input connected to the read (RD) line so it only works during read operations.

The ROM receives address lines 1 to 9 directly from the address bus. These lines are not decoded further, which means the ROM automatically gets the address range from 512 to 1023. In contrast, the RAM gets the address range from 0 to 511.

Another difference is that RAM can both read and write data, so its data bus works in both directions. But ROM is read-only, so it can only send data out, not receive it.

This example shows how memory chips are linked to the CPU. As the number of memory chips increases, the design becomes more complex, and more decoders are needed to manage which chip gets selected. That's why designers create a memory map first, which helps organize and plan the connections clearly.

Let me know if you'd like a simple diagram to visualize this setup too.

